Politecnico di Torino

Master's Degree in Cybersecurity

Lecture Notes of Post-Quantum Cryptography

Author:
Stefano Pietro Sternativo

Professor:
Nadir Murru

Academic Year: 2025/2026

Last update: 2026-06-10

Contents

Disclaimer

Notes on the handout

These notes were produced to support the Post-Quantum Cryptography course and are intended solely as a personal study and summary aid. They do not replace in any way Professor Murru's teaching activity, the lectures, the exercises done in class, or any official course material. Although I have tried to keep the text as accurate as possible, I assume no responsibility for any inaccuracies, typos, omissions, or incorrect interpretations.

I remain available for any clarification or reports at `s346910@studenti.polito.it` or on Telegram `@Spiz27`.

Custom Boxes

The document defines a few custom environments based on `tcolorbox`:

- `defBox`: green box for definitions and theoretical remarks;
- `thmBox`: blue box for theorems and formal results;
- `propBox`: violet box for propositions and observations.

Typical usage is the following:

```
\begin{defBox}{Titolo}
Text of the definition.
\end{defBox}

\begin{thmBox}{Titolo}
Text of the theorem.
\end{thmBox}

\begin{propBox}{Titolo}{label}
Text of the proposition.
\end{propBox}
```

For the `propBox` environment, two brace arguments are required: the first is the title, the second is the internal label. Even if one of them is empty, the syntax must still be written as `{}{}` to avoid compilation errors.

1 Introduction

With the development of quantum computing, quantum algorithms have been proposed to solve various problems more efficiently than classical algorithms.
One of the most famous quantum algorithms is **Shor's algorithm** for integer factorization, which can factorize large integers in polynomial time, while the best-known classical algorithms run in sub-exponential time.
Another important quantum algorithm is **Grover's algorithm** for unstructured search, which provides a quadratic speedup over classical algorithms.

Classic cryptographic protocols, such as RSA and ECC, rely on the hardness of certain mathematical problems, like integer factorization and discrete logarithms.
However, with the advent of quantum algorithms, these protocols become vulnerable to attacks performed using quantum computers.
This has led to the development of post-quantum cryptography, which aims to create cryptographic schemes that are secure against quantum attacks.

For istance, currently RSA-2048 can be broken by a quantum computer with at least ~4000 qubits.

1.1 Proposed Hard Problems

Mathematicians have been proposed new paradigms which are based on different mathematical hard problems.

1.1.1 Code based cryptography

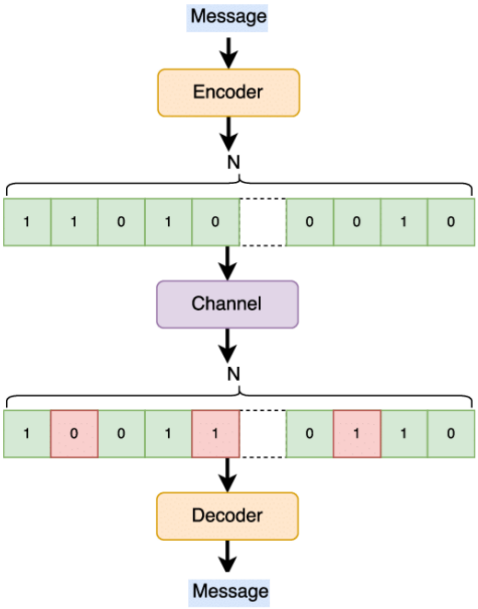


Figure 1: Code-based cryptography

Code-based cryptography is a type of post-quantum cryptography that relies on the hardness of decoding random linear codes
The most well-known code-based cryptosystem is the McEliece cryptosystem, which was proposed in 1978. It is based on the difficulty of decoding a general linear code, which is believed to be resistant to quantum attacks.

1.1.2 Lattice based cryptography

Lattice based cryptography is another approach to post-quantum cryptography that relies on the hardness of certain problems in lattice theory, such as the shortest vector problem (SVP), the closest vector problem (CVP) and the learning with errors (LWE) problem.
These problems are believed to be resistant to quantum attacks and have gained significant attention in recent years.

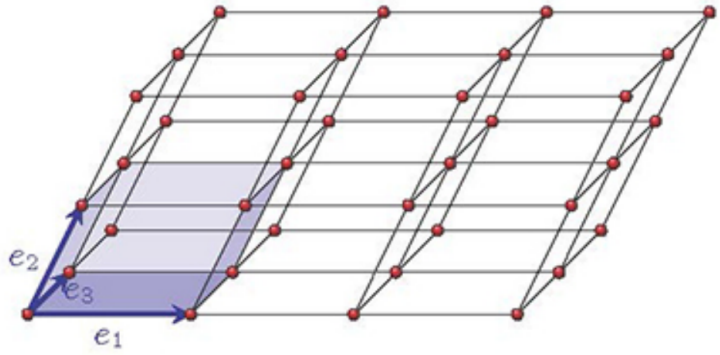


Figure 2: Lattice based cryptography

1.1.3 Multivariate cryptography

$$
\begin{aligned}
f_1(x_1, \ldots, x_n) &= \sum_{1 \le i \le j \le n} a_{ij}^{(1)} x_i x_j + \sum_{1 \le i \le n} b_i^{(1)} x_i + c^{(1)} = 0, \\
f_2(x_1, \ldots, x_n) &= \sum_{1 \le i \le j \le n} a_{ij}^{(2)} x_i x_j + \sum_{1 \le i \le n} b_i^{(2)} x_i + c^{(2)} = 0, \\
&\vdots \\
f_m(x_1, \ldots, x_n) &= \sum_{1 \le i \le j \le n} a_{ij}^{(m)} x_i x_j + \sum_{1 \le i \le n} b_i^{(m)} x_i + c^{(m)} = 0.
\end{aligned}
$$

Multivariate cryptography is a third approach to post-quantum cryptography that relies on the hardness of solving systems of multivariate polynomial equations of degree at least two.

1.2 NIST standardization

In 2017, NIST (National Institute of Standards and TechnologyUSA) opened a call for submitting new post-quntum cryptographic schemes and digital signatures, with three rounds, divided into:

- Key Encapsulation Mechanism (KEM)
- Digital Signatures

Initially, 82 proposals were submitted to the NIST PQC competition, and 69 were admitted to the first round. In July 2022, NIST announced the selected algorithms after the third round.

Selected Algorithms (Round 3)

- **Lattice-based**
  - **KEM**: CRYSTALS-Kyber
  - **Digital Signatures**: CRYSTALS-Dilithium, Falcon
- **Hash-based**
  - **Digital Signatures**: SPHINCS+

Round 4 Candidates

- **Code-based KEM**: BIKE, Classic McEliece, HQC
- **Isogeny-based**: SIKE (broken in August 2022)

New NIST Call for Digital Signatures (2024)

After the first round (October 2024), 14 signature schemes remain in the competition:

- **Lattice-based**: HAWK
- **Code-based**: CROSS, LESS
- **MPC-in-the-Head**: Mirath, MQOM, PERK, RYDE, SDitH
- **Multivariate**: MAYO, QR-UOV, SNOVA, UOV
- **Isogeny-based**: SQIsign
- **Symmetric-based**: FAEST

1.3 NOT-Covered Hard Problem

The following hard problems are not covered in this course, but they are also important in the context of post-quantum cryptography:

1.3.1 Hash based cryptography

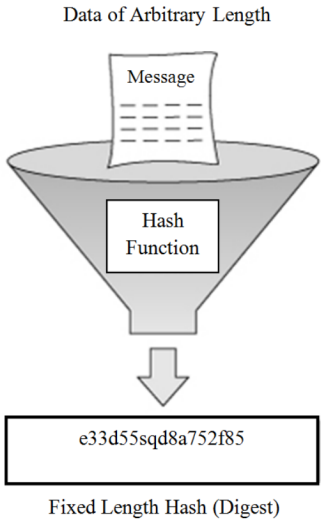


Figure 3: Hash based cryptography

Hash-based cryptography relies on the hardness of finding collisions in hash functions. It is a well-studied area of post-quantum cryptography, and several hash-based signature schemes have been proposed, such as the Merkle signature scheme and the SPHINCS.

Isogeny based cryptography

Isogeny-based cryptography relies on the hardness of finding isogenies between elliptic curves *"given two elliptic curves, finding a path between them"*. It is a relatively new area of post-quantum cryptography, and several isogeny-based schemes have been proposed, such as the Supersingular Isogeny Key Exchange (SIKE) protocol [recently broken in August 2022].

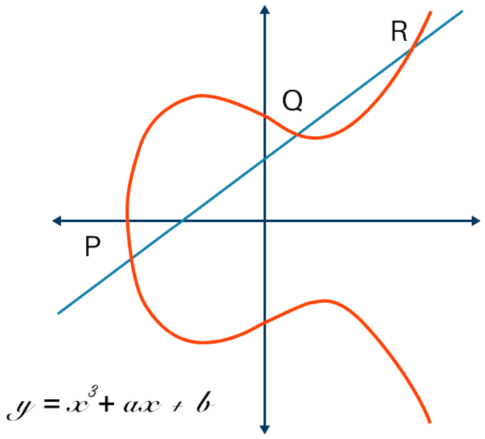


Figure 4: Isogeny based cryptography

1.4 Focus: Quantum Notation

$$|\psi\rangle = a|0\rangle + b|1\rangle, \qquad a, b \in \mathbb{C}$$

$$|a|^2 + |b|^2 = 1$$

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$|\psi\rangle = a \begin{pmatrix} 1 \\ 0 \end{pmatrix} + b \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} a \\ b \end{pmatrix} \in \mathbb{C}^2$$

$$|\psi\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

In quantum computing, the state of a qubit is represented as a linear combination of the basis states $|0\rangle$ and $|1\rangle$, where the coefficients a and b are complex numbers that satisfy the normalization condition $|a|^2 + |b|^2 = 1$. The basis states can be represented as column vectors, and the state of the qubit can be expressed as a vector in a two-dimensional complex Hilbert space.

2 Recap of Algebra

2.1 Group

1: Group

An algebraic structure $(G, *)$, where G is a set and $*$ an operation between elements of G, i.e, a function $* : G \times G \to G$, is a group if it satisfies the following properties:

- *** is associative**: $\forall g_1, g_2, g_3 \in G$, we have $(g_1 * g_2) * g_3 = g_1 * (g_2 * g_3)$.
- **Identity element**: There exists an element $e \in G$ such that $\forall a \in G$, we have $e * a = a * e = a$.
- **Inverse element**: $\forall a \in G$, there exists an element $h \in G$ such that $a * h = h * a = e$.

If $*$ is commutative, i.e., $\forall g_1, g_2 \in G$, we have $g_1 * g_2 = g_2 * g_1$, then the group is called *abelian*.

Typically, we use the following notations for groups:

Additive Notation	**Multiplicative Notation**
$(G, +)$	$(G, \cdot)$
0 is the identity	1 is the identity
$-g$ is the inverse of g	g^{-1} is the inverse of g
$Ng = \underbrace{g + \cdots + g}_{N \text{ times}}$	$g^N = \underbrace{g \cdot \cdots \cdot g}_{N \text{ times}}$

Table 1: Additive and multiplicative notation for group operations

Author's Note: The Professor underlined that it is just a notation. For istance with the 0 we don't mean the number 0, but the identity element of the group, and with $-g$ we don't mean the negative of g, but the inverse of g.

2: Cyclic Group

A group G is called *cyclic* if there exists (at least) an element $g \in G$ such that:
$G =< g >= (g)$ where $< g >= (g) = \{g^k : k \in \mathbb{Z}\} = \{kg : k \in \mathbb{Z}\}$
i.e., G is generated by g and g is called a *generator* of G.

3: Subgroup

A subgroup H of a group G is a subset of G that is itself a group under the operation of G.

Examples

- $(\mathbb{N}, +)$ is NOT a group because it lacks inverses.
- $(\mathbb{Z}, +)$ is a cyclic group generated by 1 (or -1).
- $(\mathbb{Q}, \cdot)$ is NOT a group because number 0 does not have a multiplicative inverse.

In general, we use x or $*$ for "restructing" a set to invertible elemts

- $(\mathbb{Q}^x, \cdot)$ is a group, where $\mathbb{Q}^x = \mathbb{Q} \setminus \{0\}$
- $(\mathbb{Z}^x, \cdot)$ is a group, where $\mathbb{Z}^x = \{\pm 1\}$

In cryptography it is essential know

$$\mathbb{Z}_n = \{0, 1, \ldots, n-1\}$$

that is the set of all remainders, the division by a fixed integer $N (N > 1)$.
$\mathbb{Z}_n$ is constructed starting from $\mathbb{Z}$, and introducing a relation of equalence among integers:
- given $a, b \in \mathbb{Z}$, we say that a and b are equivalent modulo N if N divides $a - b$, i.e., if there exists an integer k such that $a - b = kN$.
In this way we can partition $\mathbb{Z}$ into N equivalence classes, and each class can be represented by one of its elements, called a *representative*.

4: Invertible Element

An element $a \in \mathbb{Z}_n$ is **invertible** with respect to multiplication if and only if $gcd(a, n) = 1$.

Indeed, if $gcd(a, n) = 1$, then by the *Bezout's identity* there exist two integers x and y such that $ax + ny = 1$, and thus $ax \equiv 1 \mod n$, beacause $ny \equiv 0 \mod n$.
Consequently x is the inverse of a with respect to product.
On the other hand, if $gcd(a, n) = d \neq 1$, then a is <u>not</u> invertible with respect to the product.
Indeed, in this case we have $a = d \cdot a'$, $N = d \cdot N'$ and f we suppone by absurd that a is invertible we get a contraddiction:

$$\begin{aligned} ax &\equiv 1 \mod n \implies \\ d \cdot a'x &\equiv 1 \mod d \cdot N \implies \\ N'da'x &\equiv N' \mod N \implies \\ Na'x &\equiv N' \mod N \implies \\ 0 &\equiv N' mod N \end{aligned}$$

but $N' \neq 0$ and $N' < N$, thus we have a contraddiction.

5: Euler's Totient Function

Thus $\mathbb{Z}_n^x = \{a \in \mathbb{Z}_n : gcd(a, n) = 1\}$ and we define $\varphi(n) = |\mathbb{Z}_n^x|$ as the number of invertible elements in $\mathbb{Z}_n$.

The Euler totient function satisfies the following properties:

- If p is a prime number, then $\varphi(p) = p - 1$.
- If p is a prime number and k is a positive integer, then $\varphi(p^r) = p^r - p^{r-1} = p^r(1 - \frac{1}{p})$.
- If a and b are coprime positive integers, then $\varphi(ab) = \varphi(a) \cdot \varphi(b)$.

IMPORTANT: If we know the factorization of N, we can compute $\varphi(N)$, and viceversa.
So, compute $\varphi(N)$ is as hard as factor N.
This is the reason why $\varphi(N)$ is used in RSA, and why the security of RSA relies on the hardness of factorization.

Given $a \in \mathbb{Z}_n^x$, i.e., $gcd(a, n) = 1$, we can find its inverse usign the Bezout's identity which can be determined by means of the **Extended Euclidean Algorithm**.

The time complexity of the Extended Euclidean Algorithm is $O(\log n)$, thus we can compute the inverse of an element in $\mathbb{Z}_n^x$ in polynomial time.

Indeed, the Euclidean algo. starts computing $N = qa + r$ where $0 \leq r < N$ and we can continue until we get a remainder euqal to 0.
Thus, first of all, it is immediate to see that the time complexity is $O(a)$ or $O(N)$ since $a < N$. BUT WE CAN DO BETTER!

In fact, in a general step of the Euclidean algo. we get somthing like
$r_{i-2} = q_i r_{i-1} + r_i \geq r_i + r_{i+1}$
Now, assume that we performed t steps, then we have $r_{t+1} = 0$ and $r_t \neq 0$:

$$\begin{aligned} r_t &\geq 1 = F_1 \\ r_{t-1} &\geq r_t + r_{t+1} = r_t \geq 1 = F_2 \\ r_{t-2} &\geq r_{t-1} + r_t \geq F_2 + F_1 = F_3 \\ &\vdots \\ a > r_1 &\geq F_t \geq \phi^{t-2} \end{aligned}$$

where $\phi = \frac{1+\sqrt{5}}{2}$ is the **golden ratio**
and, where $F_1, F_2, F_3, \ldots$ are the **Fibonacci numbers**.
Thus, $\phi^{t-2} \leq a \implies \log_\phi \phi^(t-2) < \log_\phi a \implies t - 2 < \log_\phi a \implies t \sim O(\log a) \sim O(\log N)$.

So, by previous explanations, we can determinate that: $(\mathbb{Z}_N, +)$ is a cycling group, indeed 1 is a generator: $< 1 >= \mathbb{Z}_N$
All the generators of $\mathbb{Z}_N$ are teh elements $a \in \mathbb{Z}_N$ such that $gcd(a, N) = 1$:
$< a >= \{ka \in \mathbb{Z}_N : k \in \mathbb{Z}\} = \mathbb{Z}_N$ where $ka = \frac{a+\ldots+a}{k}$

Example
Consider $(\mathbb{Z}_{12}, +)$:

- $< 1 >= \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\} = \mathbb{Z}_{12}$
- $< 2 >= \{2, 4, 6, 8, 10, 0\} \subseteq \mathbb{Z}_{12}$
- $< 3 >= \{3, 6, 9, 0\} \subseteq \mathbb{Z}_{12}$
- $< 4 >= \{4, 8, 0\} \subseteq \mathbb{Z}_{12}$
- $< 5 >= \{5, 10, 3, 8, 13, 6, 11, 4, 9, 2, 7, 0\} \subseteq \mathbb{Z}_{12}$

By analyzing the last example (<5>), we can see that $5 \equiv 0 \mod 12$, so we can generalize that:
$5k \equiv 0 \mod 12 \iff k$ is multiple of 12.

Remark
The number oof generators of $\mathbb{Z}_N$ is $\varphi(N)$.

6: Morphism

Given a function $f : (G, *) \to (G', *')$, then f is a morphism if $\forall g_1, g_2 \in G$, we have $f(g_1 * g_2) = f(g_1) *' f(g_2)$.

A biejective morphism is called an **isomorphism**.

1: Isomorphism of Cyclic Groups

- A cycling group with infintely many elements is isomorphic to $(\mathbb{Z}, +)$.
- A cycling group with N elements is isomorphic to $(\mathbb{Z}_N, +)$.

The idea of proof:
Consider a cyclic group G with infintely many elements:
Let g be a generator of G, then we can define a function $f : \mathbb{Z} \to G$ such that $f(k) = kg$.
It is easy to see that f is a morphism, indeed $f(k_1 + k_2) = (k_1 + k_2)g = k_1g + k_2g = f(k_1) + f(k_2)$.
Moreover, f is injective, indeed if $f(k_1) = f(k_2)$, then $k_1g = k_2g$, and since g is a generator, we have $k_1 = k_2$.
Finally, f is surjective, indeed for every $h \in G$, there exists $k \in \mathbb{Z}$ such that $h = kg$, thus $f(k) = h$.

Remark:
$(\mathbb{Z}_N^x, \cdot)$ is a cycling group $\iff N = 2, 4, p^k, 2p^k$ where p is an odd prime and $k \geq 1$.
Thus, in this case we have $(\mathbb{Z}_N^x, \cdot) \simeq (\mathbb{Z}_{\varphi(N)}, +)$.

7: Kernel

Given a morphism $f : (G, *) \to (G', *')$, the kernel of f is the set of elements of G that are mapped to the identity element of G', i.e., $\text{Ker}(f) = \{g \in G : f(g) = e'\}$ where e' is the identity element of G'.

Author's Note: I use the letter e (for semplicity) for the identity element of G. If we want to follow the Murru's notation, we should use 1_G

1: Kernel injectivity

f is injective $\iff$ $\text{Ker}(f) = \{e\}$ where e is the identity element of G.

8: Subgroup Characterization

Let G be a group, then $H \subseteq G$ is a subgroup of G if and only if H is a group with respect to the operation of G.

2: Subgroup Test

$H \subseteq G$ is a subgroup $\iff$ $\forall h_1, h_2 \in H$, we have $h_1 * h_2^{-1} \in H$.

Example:
Consider $G = (\mathbb{Z}, +)$, then the subgroups of G are the following set:
$< N >= (N) = N\mathbb{Z} = \{kN : k \in \mathbb{Z}\}$ (the set of all multiples of N)

Indeed, given $h_1, h_2 \in N\mathbb{Z}$, we have that $h_1 - h_2 = k_1N - k_2 - N = (k_1 - k_2)N \in N\mathbb{Z}$.
Thus, $N\mathbb{Z}$ is a subgroup of $(\mathbb{Z}, +)$. On the other hand, any subset H of $\mathbb{Z}$ is of the kind $N\mathbb{Z}$.
Consider $H \subseteq \mathbb{Z}$ with $H \neq \{0\}$, thus H must contain at least one positive integer N.
Let N be the smallest positive integer in H (no zero), then we have that $N\mathbb{Z} \subseteq H$, because H must be closed under $+$.
Now, consider $m \in H$ and let us divide it by N: $m = qN + r$ with $0 \leq r < N$.
$\implies r = m - qN \in H \implies r = 0$ (becauuse N is the smallest positive integer in H).
$\implies m$ is a multiple of N, thus $H \subseteq N\mathbb{Z}$.

2: Lagrange's Theorem

Given a finite group $(G, *)$ and a subgroup $H \leq G$, then $|H| \mid |G|$.
Where $|H|$ and $|G|$ are the number of elements of H and G respectively.

Corollary:
If $|G| = n$, then $\forall g \in G$, we have $g^n = 1$ or $ng = 0$ depending on the notation we use for the group operation.

3: Euler's Theorem

$a \in \mathbb{Z}_N^x$, then $a^{\varphi(N)} \equiv 1 \mod N$.

4: Fermat's Little Theorem

$a \in \mathbb{Z}_p^x$, p prime, then $a^{p-1} \equiv 1 \mod p$.

2.2 Rings

9: Rings

Given an algebraic structure $(R, *_1, *_2)$,then R is a ring if:

- $(R, *_1)$ is an abelian group.
- $*_2$ is associative, $*_2$ is commutative (only for our purposes), and $*_2$ has an identity element with respect to $*_2$.
- Distrubutive property: $\forall a, b, c \in R$, we have $a *_2 (b *_1 c) = (a *_2 b) *_1 (a *_2 c)$ and $(b *_1 c) *_2 a = (b *_2 a) *_1 (c *_2 a)$.

Usually we use the following notations for rings: $(R, +, \cdot)$

Examples:

- $(\mathbb{Z}, +, \cdot)$ is a ring of integers.
- $(\mathbb{Z}_N, +, \cdot)$ is a ring of resuders modulo N.

For the groups, we have constructed $\mathbb{Z}_N$ starting from $\mathbb{Z}$, and introducing a relation of equivalence among integers.
In particular, we verified that $a \sim b \iff a \equiv b \mod N$.

We can provide the same construction considering the ring $(\mathbb{Z}, +, \cdot)$ and the same relation of equivalence, that will determinate the new ring $(\mathbb{Z}_N, +, \cdot)$ (we can aslo write $\mathbb{Z}_N \simeq \mathbb{Z}/N$)

When we introduce a relation of equivalence, we are also providing a partition of the set $\mathbb{Z}$ into equivalence classes, and each class can be represented by one of its elements, called a *representative*.
$[0]$ is the class of all integers with remainder $0 : ... - 2N, -N, 0, N, 2N, ...$
$[1]$ is the .... integers with remainder $1 : ... -2N+1, -N+1, 1, N+1, 2N+1, ...$
$[0] \cap [1] = \emptyset$ and $[0] \cup [1] \cup ... \cup [N-1] = \mathbb{Z}$

We can obtain $\mathbb{Z}_N$ starting from $\mathbb{Z}$ in a differten (equivalent) way.
Consider the ring $\mathbb{Z}, +, \cdot$, the following set $< N >= (N) = N\mathbb{Z} = \{kN : k \in \mathbb{Z}\}$ (the set of all multiples of N) are a special subset of $\mathbb{Z}$, that satisfy the following properties:

- (N) is a subgroup of $(\mathbb{Z}, +)$

- $\forall a \in \mathbb{Z}$, $\forall b \in (N)$, we have $ab \in (N)$ where (N) is the ideal of $(\mathbb{Z}, +, \cdot)$

If we fix an ideal (N), then we can derive a relation of equivalence:
$a, b \in \mathbb{Z}$ are equivalent $a \sim b \iff a + (-b) \in (N)$ (this condition is the same as $a \equiv b \mod N$).

Then, $\mathbb{Z}_N$ is the set od class of equivalence with respect to this relation, and we can write $\mathbb{Z}_N = \mathbb{Z}/(N) = \mathbb{Z}_N/N\mathbb{Z}_N$

2.3 Field

10: Field

Given an algebraic structure $(F, +, \cdot)$, then F is a field if:

- $(F, +)$ is an abelian group.
- $(F \setminus \{0\}, \cdot)$ is an abelian group where 0 is the identity element of $(F, +)$.
- Distrubutive property: $\forall a, b, c \in F$, we have $a \cdot (b+c) = a \cdot b + a \cdot c$ and $(b + c) \cdot a = b \cdot a + c \cdot a$.

When a field has a finitely many elements, we call it a **finite field** or *Galois field*.

Examples:
$(\mathbb{Z}_p[X], +, \cdot)$ is a ring, then $(f(X)) = \{g(x)f(x) : g(x) \in \mathbb{Z}_p[X]\}$
$f(x)$ is invertible then $\mathbb{Z}_p[X]/(f(X))$ is a field

11: Ideal

Let $(R, +, \cdot)$ be a ring, then a subset $I \subseteq R$ is an ideal if:

- $(I, +)$ is a subgroup of $(R, +)$
- $\forall r \in R$, $\forall i \in I$, we have $ri \in I$ and $ir \in I$.

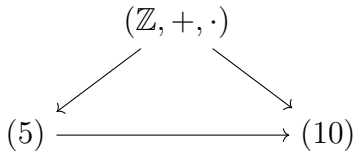
Given an ideal, it is possible to define the following relation of equivalence: $a, b \in R$ are equivalent $a \sim b \iff a + (-b) \in I$, and we denote by R/I
This set is stilla a ring.

Example:
Consider the ring $(\mathbb{Z}, +, \cdot)$. The sets $(n) = n\mathbb{Z} = \{kn : k \in \mathbb{Z}\}$ are ideals of $\mathbb{Z}$.

Thus $\mathbb{Z}/(n) = \mathbb{Z}/n\mathbb{Z} = \mathbb{Z}$ is a new ring.
In this case, the relation of equivalence is:
$\forall a, b \in \mathbb{Z}, a \sim b \iff a + (-b) \in (n) \iff a - b = kn$ for some $k \in \mathbb{Z} \iff a \equiv b \mod n$

If p is a prime number, the (p) is a special ideal called **maximal ideal**, (maximal ideal is an ideal that is not contained in any other ideal) and $\mathbb{Z}/(p) = \mathbb{Z}/p\mathbb{Z}$ is a field.

Example:
For example (5) is a maximal ideal, while (10) is not a maximal ideal, because (5) is an ideal that contains (10):



Because

$$(5) = \{5k : k \in \mathbb{Z}\} = \{..., -10, -5, 0, 5, 10, ...\}$$
$$(10) = \{10k : k \in \mathbb{Z}\} = \{..., -20, -10, 0, 10, 20, ...\}$$

and $(10) \subseteq (5)$ but (5) is not contained in any other ideal.

5: Galois's Theorem on Finite Fields

Let $\mathbb{F}_q$ be a finite field with q elements, then we can only have the following situations:

- $q = p$ where p is a prime number, and in this case $\mathbb{F}_q \simeq \mathbb{Z}_p$, so they are isomorphic.
- $q = p^k$ where p is a prime number and k is a positive integer, and in this case $\mathbb{F}_q \simeq \mathbb{F}_p[X]/(f(X))$ where $f(X)$ is an irreducible polynomial of degree k over $\mathbb{F}_p$.

12: Irreducible Polynomial

Given a field $\mathbb{F}$, a polynomial $f(X) \in \mathbb{F}[X]$ is irreducible if it cannot be factorized into the product of two polynomials of degree >0 and <deg(f(x)).

Remark:
Irreducible polynomials play the same role for polynomials as prime numbers play for integers.

Given the ring $(\mathbb{Z}_p[x], +, \cdot)$ all and only the ideal are the sets
$(f(x)) = \{g(x)f(x) : g(x) \in \mathbb{Z}_p[x]\}$ = multiples of $f(x)$
so, they are the analogue of the ideals (n) for $\mathbb{Z}$

If $f(x)$ is irreducible, then $(f(x))$ is a maximal ideal and $\mathbb{Z}_p[X]/(f(x))$ is a finite field with p^k elements, where k is the degree of $f(x)$.

Indeed, this is the set of classes of equivalence defined by
$\forall g_1(x), g_2(x) \in \mathbb{Z}_p[X], g_1(x) \sim g_2(x) \iff g_1(x) + (g_2(x)) \in (f(x)) \iff$
$g_1(x) - g_2(x) = g(x)f(x)$ for some $g(x) \in \mathbb{Z}_p[X] \iff g_1(x)$ and $g_2(x)$ have the same remainder with respect to the division by $f(x) \iff g_1(x) \equiv g_2(x) \mod f(x)$

Thus $\mathbb{Z}_p[X]/(f(x))$ is the set of all possible reminders with respect to the division by $f(x)$, which are polynomials of degree $< \deg(f(x))$.
Indeed, given $h(x) \in \mathbb{Z}_p[X]$, we can write $h(x) = q(x)f(x) + r(x)$ where $r(x)$ is the remainder and $\deg(r(x)) < \deg(f(x))$.

Thus a reminder, i.e., an element of $\mathbb{Z}_p[X]/(f(x))$, is a polynomial of kind $a_0 + a_1x + \cdots + a_{k-1}x^{k-1}$ where $a_i \in \mathbb{Z}_p$.
Thus we have p^k possible reminders, this is, p^k possible elements in $\mathbb{Z}_p[X]/(f(x))$.

Remark:
Let $\mathbb{F}$ be a field:

- if $f(x) \in \mathbb{F}[X]$ has a root $\alpha \in \mathbb{F}$ (i.e., $f(\alpha) = 0$), then $f(x)$ is not irreducible (it is irreducible). (*)
- if $f(x) \in \mathbb{F}[X]$ has degree 2 or 3 and has no root in $\mathbb{F}$, then $f(x)$ is irreducible (if $\deg(f(x)) > 3$ and f has no roots, we can not say anything)

(*) Remember that if α is a root of $f(x)$, then $f(x) = (x - \alpha)g(x)$

3: Irreducibility

A polynomia $f(x) \in \mathbb{Z}_p[X]$ of degree k is irreducible if and only if:

- $f(x) | x^{p^k} - x$
- $\forall$ prime $r | k, gcd(f(x), x^{p^{k/r}} - x) = 1$

This proposition works well if k is small (and it can be used very often in the construction of finite fields).
In general, it is not efficient because we need to know the factorization of k. But there exist some efficient test for checking the irreducibility of a polynomial.

BIG EXAMPLE: Construction of $\mathbb{F}_4$

$\mathbb{F}_4 = \mathbb{F}_2$, thus we need to construct $\mathbb{F}_2[X]/(f(X))$ where $f(X)$ is an irreducible polynomial of degree 2 over $\mathbb{F}_2$.

Remember that we are working in $\mathbb{F}_2[X]$, thus a polynomial of degree 2 is of the kind $ax^2 + bx + c$ where $a, b, c \in \mathbb{F}_2$. Thus, a can be only equal to 1 (otherwise the polynomial does not have degree 2)
$b, c \in \{0, 1\}$, thus we have 4 possible polynomials of degree 2:

$x^2, x^2 + 1, x + 1, x^2 + x + 1$

- x^2 is not irreducible because it has a root (0)
- $x^2 + 1 = (x + 1)(x + 1) = (x + 1)^2$ is not irreducible (1 is a root)
- $x^2 + x = x(x + 1)$ is not irreduucible (0 and 1 are roots)

- $x^2 + x + 1$ is irreducible (it has no roots in $\mathbb{F}_2$)

Thus $\mathbb{F}_4 = \mathbb{F}_2[X]/(x^2 + x + 1)$ = reminders with respect to the division by $x^2 + x + 1$ = all polynomials $\in \mathbb{F}_2[X]$ of degree $< 2 = \{0, 1,$ [poly of deg 0] $x, x + 1$[poly of deg 1] $\}$
$\mathbb{F}_4 = \{0, 1, x, x + 1\} = \{0, 1, \alpha, \alpha + 1\}$ where α is a root in $\mathbb{F}_2$ of $x^2 + x + 1$ (somewhere else!)

Let us observe that x and α have the same behavior.
Indeed, $x^2 + x + 1$ is irreducible in $\mathbb{F}_4$, becasue $x^2 = 1(x^2 + x + 1) + x + 1$ where 1 is the quotient and $x + 1$ is the reminder.
Indeed, $x^2 + x + 1 + x + 1 = x^2 + 2x + 2 = x^2$ because we are working in $\mathbb{F}_2$ and $2x = 0$ and $2 = 0$.

On the other hand, it is possible to observe that:
$x^2 \equiv x^2+x^2+x+1 \mod x^2+x+1 \implies x^2 \equiv 2x^2+x+1 \mod x^2+x+1 \implies x^2 \equiv x + 1 \mod x^2 + x + 1$
Similarly, since α is a root of $x^2 + x + 1$, we have $\alpha^2 + \alpha + 1 = 0 \implies \alpha^2 = -\alpha - 1 \implies \alpha^2 = \alpha + 1$.
End example
other practial examples have been provided at lecture of 13-03-2026

13: Primitive

Given $f(x) \in \mathbb{F}_p[X]$ if $f(x)$ is irreducible and
$< x >= (\mathbb{F}_p[X]/(f(x)))^2$ then $f(x)$ is called primitive.

6: Cyclicality of Finite Fields

Given a finite field $\mathbb{F}_q$, then $(\mathbb{F}_q^x, \cdot)$ is a cyclic group.

These concepts are strictly related to the cryptographic applications, and they will be used in the construction of the AES (Advanced Encryption Standard) and in the construction of the elliptic curves.

For istance, **AES:**
It is possible see AES consideing the fact that *"bites are strings of bites of lenght 8. The number of possivle bytes is* 2^8 *"*
So $\mathbb{F}_2^8 = \mathbb{F}_2 \cdot \mathbb{F}_2 \cdots \mathbb{F}_2$ (8 times) VECTOR SPACE, but we can also see it as a field, because we can find an irreducible polynomial of degree 8 over $\mathbb{F}_2$,

thus we can write
$\mathbb{F}_{2^8} = \mathbb{F}_2[X]/(f(X))$ where $f(X)$ is an irreducible polynomial of degree 8 over $\mathbb{F}_2$.
$a_7x^7 + a_6x^6 + \cdots + a_1x + a_0$ where $a_i \in \mathbb{F}_2$ is the representation of a byte in AES (0, 1), and the operations of AES are defined as the operations of $\mathbb{F}_2[X]/(f(X))$.

In AES $\mathbb{F}_{2^8} \simeq \mathbb{F}_2[X]/(x^8 + x^4 + x^3 + x + 1)$

> ***Author's Note:*** *The Professor underlined that it is important, during individual study, to consider the difference between the two representations of $\mathbb{F}_{2^8}$, because they are equivalent but they have different properties.*
> *In particular, the representation $\mathbb{F}_2^8$ is a vector space, thus it has a basis and we can represent the elements of $\mathbb{F}_{2^8}$ as linear combinations of the basis elements, while the representation $\mathbb{F}_2[X]/(f(X))$ is a field, thus it has a multiplicative structure and we can find the inverse of an element with respect to the multiplication.*

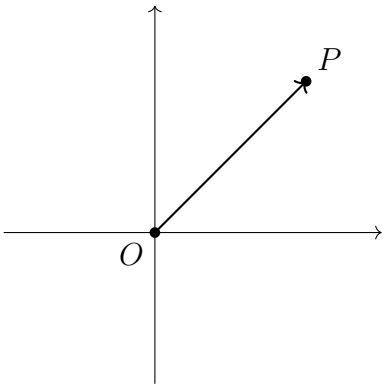
3 Linear Algebra

Linear algebra is a branch of mathematics that studies vectors, vector spaces, and linear transformations, so linear objects.
It provides the foundation for many areas of mathematics and is essential in various applications, including cryptography.

For example, cosider $\mathbb{R}^2 = \mathbb{R}$ x $\mathbb{R} = \{(x, y) : x, y \in \mathbb{R}\}$
In $\mathbb{R}^2$, Euclidean plane, the most simple linear object are point or equivalently vectors.

3.1 Vector

Indeed, a point in $\mathbb{R}^2$ is indentified by pairs of real numbers. Indeed a vector is an "arrow" strating from the origin, thus a point, i.e., a pair of real numbers, indetifies also a vector.



Formally:

14: Vector

A vector $v \in \mathbb{R}^n$ is a n-tuple of real numbers

$$v = (v_1, v_2, ..., v_n)$$

With following operations:

- **Multiplication by a scalar**: $tv = (tv_1, tv_2, ..., tv_n)$ for $t \in \mathbb{R}$ and $v \in \mathbb{R}^n$
- **Sum**: $v + w = (v_1 + w_1, v_2 + w_2, ..., v_n + w_n)$ for $v,w \in \mathbb{R}^n$
- **Dot product**: $v \cdot w = v_1w_1 + v_2w_2 + ... + v_nw_n$ for $v,w \in \mathbb{R}^n$
- **Norm**: $\|v\| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$ for $v \in \mathbb{R}^n$

Example:
Let v = (2,-1,5) and w = $(\sqrt{2},\frac{1}{3},0)$ be two vectors in $\mathbb{R}^3$.
Then:

- $\frac{1}{2}v = (1, -\frac{1}{2}, \frac{5}{2})$
- $v + w = (2 + \sqrt{2}, -\frac{2}{3}, 5)$
- $\|w\| = \sqrt{2 + \frac{1}{9} + 0} = \sqrt{\frac{19}{9}} = \frac{\sqrt{19}}{3}$

15: Linear Independence

Given k vectors $a_1, a_2, ..., a_k$ in $\mathbb{R}^n$, they are **linearly dependent** if there exist scalars $t_1, t_2, ..., t_k$, not all zero, such that:

$$t_1a_1 + t_2a_2 + ... + t_ka_k = 0$$

Otherwise, they are **linearly independent**.
This means that, if $a_1, a_2, ..., a_k$ are linearly independent, then we can write one of them as a linear combination of the others one

This peculiarity is fundamental in cryptography, indeed, for example, in the case of the **LWE problem**, we have a system of linear equations, and the security of the scheme relies on the fact that these equations are linearly independent, thus we cannot solve them easily by simplify them one with the other much simpler.

Example:
Let us consider $a_1 = (1, 0, -1, 1)$, $a_2 = (2, 1, 0, -1)$, $a_3 = (1, -1, -3, 4)$, $a_4 =$

(2, 5, 0, -1).For instance, we have that a_1, a_2 and a_3 are linearly independent, while a_4 is linearly dependent on the others.
So:

$$3a_1 - a_3 - a_4 = 0$$

and

$$a_4 = 3a_1 - a_3$$

On the other hand, a_1, a_2 and a_4 are linearly independent, because:

$$xa_1 + ya_2 + za_4 = (x + 2y + 2z, y + 5z, -x, x - y - z) = (0, 0, 0, 0)$$

if and only if $x = y = z = 0$, thus they are linearly independent.

3.2 Lines

A **line** in $\mathbb{R}^n$ can be defined as the set of points that can be expressed as a linear combination of a fixed point and a direction vector.
Considering a line in $\mathbb{R}^2$, it is a subset of $\mathbb{R}^2$ that can be expressed as:

$$y = mx + q \text{ with } m, q \in \mathbb{R}$$

A linear can be also described in a parametric form as:

$$\begin{cases} x = tv_1 + x_A \\ y = tv_2 + y_A \end{cases}$$

This is the line through the point $A = (x_A, y_A)$ and with direction the vector $v = (v_1, v_2)$.

Linear objects in higher dimensions

Similarly, in $\mathbb{R}^3$, we have the following linear object:

- **Point/vectors**
- **Lines** –> described by two linear equations

$$\begin{cases} ax + by + cz + d = 0 \\ a^i x + b^i y + c^i z + d^i = 0 \end{cases}$$

- **Planes** –> described by one linear equation $ax + by + cz + d = 0$

These are only some of the linear objects, and we can also consider higher-dimensional linear objects.

3.3 Matrix

A **matrix** is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns.
Formally:

16: Matrix

Given m,n $\in \mathbb{N}$, a matrix A of size m x n is a table of mn real numbers, arranged in m rows and n columns.

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Where $a_{ij} \in \mathbb{R}$ for all i = 1, 2, ..., m and j = 1, 2, ..., n.

Sometimes, we write matrix using the following notation: $A = (a_{ij})$.

Classification:

- **Square matrix**: if m = n
- **Row vector**: if m = 1, so $\mathbb{R}^{1\times n}$
- **Column vector**: if n = 1, so $\mathbb{R}^{m\times 1}$

Operations on matrices:

- **Multiplication by a scalar:** $tA = (ta_{ij})$ for $t \in \mathbb{R}$ and $A \in \mathbb{R}^{m\times n}$
- **Sum:** $A + B = (a_{ij} + b_{ij})$ for $A, B \in \mathbb{R}^{m\times n}$
- **Transpose:** $A^T = (a_{ji})$ for $A \in \mathbb{R}^{m\times n}$, obteined by swapping rows and columns
- **Product:** $AB = (c_{ij}) \in \mathbb{R}^{m\times r}$, for all $A \in \mathbb{R}^{m\times n}$ and $B \in \mathbb{R}^{n\times r}$, where the elements c_{ij} is obteined by performing the dot product between the i-th row of A and the j-th column of B

Example:

Let A = $\begin{pmatrix} 1 & 2 & 3 \\ 0 & 1 & -1 \end{pmatrix} \in \mathbb{R}^{2\times 3}$ and B = $\begin{pmatrix} 2 & 0 \\ 1 & 1 \\ -1 & 3 \end{pmatrix} \in \mathbb{R}^{3\times 2}$

Then AB = $\begin{pmatrix} 1 & 11 \\ 2 & -2 \end{pmatrix} \in \mathbb{R}^{2\times 2}$, BA = $\begin{pmatrix} 2 & 4 & 6 \\ 1 & 3 & 2 \\ -1 & 1 & -6 \end{pmatrix} \in \mathbb{R}^{3\times 3}$

Given C = $\begin{pmatrix} 5 & \sqrt{2} & \frac{1}{2} & 0 \\ -1 & 2 & \pi & -3 \end{pmatrix} \in \mathbb{R}^{2\times 4}$, its trasponse is $C^T = \begin{pmatrix} 5 & -1 \\ \sqrt{2} & 2 \\ \frac{1}{2} & \phi \\ 0 & -3 \end{pmatrix} \in$ $\mathbb{R}^{4\times 2}$.

17: Identity matrix

The identity matrix I_n of size n x n is the square matrix with 1's on the main diagonal and 0's elsewhere.

$$I_n = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix}$$

Important notions

The **determinant** of a square matrix can be evaluted recursively.

Given a 2 x 2 matrix A = $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$, its determinant is det(A) = ad - bc.

Given a 3 x 3 matrix B = $\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$,

its determinant is det(B) = a det $\begin{pmatrix} e & f \\ h & i \end{pmatrix}$ - b det $\begin{pmatrix} d & f \\ g & i \end{pmatrix}$ + c det $\begin{pmatrix} d & e \\ g & h \end{pmatrix}$ = a(ei - fh) - b(di - fg) + c(dh - eg).

Example:

Given A = $\begin{pmatrix} 2 & 0 & 1 & -3 \\ 1 & 5 & -1 & 2 \\ 4 & 1 & -2 & 7 \\ 0 & 0 & 2 & 0 \end{pmatrix}$ then, det(A) = -2 det $\begin{pmatrix} 2 & 0 & -3 \\ 1 & 5 & 2 \\ 4 & 1 & 7 \end{pmatrix}$ and

det $\begin{pmatrix} 2 & 0 & -3 \\ 1 & 5 & 2 \\ 4 & 1 & 7 \end{pmatrix}$ = 2 det $\begin{pmatrix} 5 & 2 \\ 1 & 7 \end{pmatrix}$ - 3 det $\begin{pmatrix} 1 & 5 \\ 4 & 1 \end{pmatrix}$

The **trace** of a square matrix A = (a_{ij}) is the sum of the elements on the main diagonal, i.e., tr(A) = a_{11} + a_{22} + ... + a_{nn}.

Example:
Given

$$A = \begin{pmatrix} 2 & -1 & 0 \\ 4 & 3 & 5 \\ 1 & 2 & -2 \end{pmatrix}$$

the trace is

$$\mathrm{tr}(A) = 2 + 3 + (-2) = 3.$$

3.3.1 Matrix multiplication algorithms

The standard algorithm for multiplying two n x n matrices has a time complexity of O(n^3).

Algorithm 1 Schoolbook Matrix Multiplication

Require: Matrices $A \in \mathbb{R}^{n \times m}$, $B \in \mathbb{R}^{m \times p}$
Ensure: Matrix $C \in \mathbb{R}^{n \times p}$

```
Initialize C as a zero matrix
for i = 1 to n do
    for j = 1 to p do
        sum ← 0
        for k = 1 to m do
            sum ← sum + A_ik · B_kj
        end for
        C_ij ← sum
    end for
end for
return C
```

However, other algorithms exist:

- **Strassen's algorithm** reduces the time complexity to O($n^{2.807}$).

- **Coppersmith-Winograd algorithm** further reduces the time complexity to $O(n^{2.376})$.

- **Optimized CW-like algorithms** have been developed, achieving time complexities of $O(n^{2.373})$

3.3.2 Matrix inversion algorithms

A matrix is invertible if its determinant is non-zero.
The standard algorithm for inverting an n x n matrix (**Gauss-Jordan elimination**)has a time complexity of $O(n^3)$.

Algorithm 2 Gauss-Jordan

1: Swapping two rows
2: Multiplying a row by a non-zero scalar
3: Adding a multiple of one row to another row

However, other algorithms exist:

- **Strassen's algorithm** can be adapted for matrix inversion, achieving a time complexity of $O(n^{2.807})$.

- **Coppersmith-Winograd algorithm** can also be adapted for matrix inversion, achieving a time complexity of $O(n^{2.376})$.

- **Optimized CW-like algorithms** have been developed for matrix inversion, achieving time complexities of $O(n^{2.373})$.

Professor Murru underlines that these algorithms are indeed polynomial-time. However, this can be misleading: in this context, the complexity is expressed in terms of the matrix dimension n, not in terms of the bit-length of the input.

When considering the actual input size (i.e., the number of bits needed to represent the matrices), the complexity may become much larger in practice. Therefore, although these algorithms are polynomial in n, they are not necessarily efficient in the sense typically considered in cryptography, where efficiency is measured with respect to the bit-size of the input.

3.3.3 Determinant algorithms

The determinant of an n x n matrix can be computed using the standard algorithm (**Laplace expansion**) with a time complexity of O(n!).
However, more efficient algorithms exist:

- **LU decomposition** can be used to compute the determinant in $O(n^3)$ time.
- **Division-free algorithms** can compute the determinant in $O(n^3)$ time without performing any division, which is particularly useful for integer matrices.
- **Fast matrix multiplication** can compute the determinant in $O(n^{2.373})$ time, leveraging fast matrix multiplication techniques.
- **Bareiss algorithm** is an efficient method for computing the determinant with a time complexity of $O(n^3)$ and is particularly useful for integer matrices, as it avoids intermediate fractions.

Algorithm 3 Bareiss Algorithm

Input: M — an $n \times n$ matrix, with non-zero leading principal minors
Initialize: $M_{0,0} \leftarrow 1$
for $k = 1$ to $n - 1$ **do**
    for $i = k + 1$ to n **do**
        for $j = k + 1$ to n **do**
            $M_{i,j} \leftarrow \frac{M_{i,j}M_{k,k} - M_{i,k}M_{k,j}}{M_{k-1,k-1}}$
        end for
    end for
end for
Output: $M_{n,n}$ contains $\det(M)$

3.4 Vector Space

18: vector space

A **Vector space** over the real (or complex) numbers is a set V where there exists an operation between the elements of V that we denote by + and an operation between elements of V and elements of $\mathbb{R}$ (or $\mathbb{C}$) called multiplication by scalar, having good properties:

- + must be associative, there exists the identity, there exists the inverse, and it must be commutative
- Scalar multiplication must be associative and distributive with respect to the sum

Remark:
The set $\mathbb{R}^n$ of vectors is a vector space over $\mathbb{R}$. The set $\mathbb{R}^{m\times n}$ of matrices is a vector space over $\mathbb{R}$, but we can also consider the set of matrices as a vector space over $\mathbb{C}$.

Be careful: $\mathbb{R}^n$ is a vector space over $\mathbb{C}$, but it is not a vector space over $\mathbb{F}_q$, where $\mathbb{F}_q$ is a finite field with q elements, because the scalar multiplication is not defined for all scalars in $\mathbb{F}_q$.

Consider, e.g., $\mathbb{R}^3$ and $\mathbb{F}_5$ and let see that $\mathbb{R}^3$ is not a vector space over $\mathbb{F}_5$. Indeed, if we take the "scalar" $0\in \mathbb{F}_5$, and a non-zero vector $v \in \mathbb{R}^3$ (i.e. v = (1, 2, 3)), then we have that $0\cdot$ v = (0, 0, 0) and since 5=0 in $\mathbb{F}_5$, we have that $5\cdot$ v = 0

On the other hand, we can computer $5 \cdot$ v also in the following way: $5\cdot$ v = $(1 + 1 + 1 + 1 + 1)$v = v + v + v + v + v = (1, 2, 3) + ...+ (1, 2, 3) = (5, 10, 15) $\neq$ (0, 0, 0) $\in \mathbb{R}^3$.
because 5,10,15 are elements of $\mathbb{R}$, are not 0.

Similarly, we can consider $\mathbb{F}_q^n$ as a vector space over $\mathbb{F}_q$, but we cannot consider $\mathbb{F}_q^n$ as a vector space over $\mathbb{R}$, because the scalar multiplication is not defined for all scalars in $\mathbb{R}$.
The main reason (without formal proof) is that:

- $\mathbb{F}_q, q = p^k$ where p is primem has characteristic p, wich means that $\forall\alpha \in \mathbb{F}_q$, we have $\alpha + ... + \alpha$ (p times) = 0
- $\mathbb{R}$ has characteristic 0, wich means that $\forall\alpha \in \mathbb{R} \setminus \{0\}, \alpha + ... + \alpha$ is always different from 0

3.4.1 Basis

In a vector space, there always exist a basis.

19: Basis

A basis is a set of vector that generate all the other ones by means of their linear combination.

The vectors of a basis are linearly independent and they are set of generators of the vector space.
The number of vectors in a basis is called the **dimension** of the vector space.

Example
In $\mathbb{R}^3$, the standard basis is given by the vectors $e_1 = (1, 0, 0)$, $e_2 = (0, 1, 0)$ and $e_3 = (0, 0, 1)$.

3.4.2 Linear map

A **linear map** (or linear transformation) is a function between two vector spaces that preserves the operations of vector addition and scalar multiplication.

$$f : V \to W$$

where V and W are vector spaces , susch that for all u, v $\in V$ and t $\in \mathbb{R}$ (or $\mathbb{C}$), we have:

$$f(v_1 + v_2) = f(v_1) + f(v_2), f(kv) = kf(v)$$

A linear map can be represented by a matrix f(v) = Av, where A is a matrix and v is a vector.

Example:
Consider f: $\mathbb{R}^2 \to \mathbb{R}^3$, such that

$$f(v) = w, w = (v_1 + v_2, v_3, 2v_4 - v_1, v_2 + 3v_3)$$

This is a linear map, and it can be represented by the following matrix:

$$A = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 1 \\ 0 & 1 & 3 & 0 \end{pmatrix}$$

3.4.3 Eigenvalues and eigenvectors

Given a matrix A, or equivalently a linear map $f : V \to V$, a non-zero vector v is called an **eigenvector** of A if there exists a scalar λ such that Av = λv. The scalar λ is called the **eigenvalue** corresponding to the eigenvector v.

To compute the eigenvalues and eigenvectors of a matrix A, we can solve the characteristic equation det(A - λI) = 0, where I is the identity matrix.
In this way, we can find the eigenvalues λ, and then we can find the corresponding eigenvectors by solving the equation (A - λI)v = 0 for each eigenvalue λ.

3.4.4 Tensor product

The **tensor product** is a way to combine two vector spaces V and W into a new vector space V $\otimes$ W.

20: Tensor product

Given two matrices A $\in \mathbb{R}^{m \times n}$ and B $\in \mathbb{R}^{p \times q}$, then their **Kronocker product** is:

$$A \otimes B = \begin{pmatrix} a_{11}B & a_{12}B & \cdots & a_{1n}B \\ a_{21}B & a_{22}B & \cdots & a_{2n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}B & a_{m2}B & \cdots & a_{mn}B \end{pmatrix}$$

4 Coding Theory

Coding theory is a branch of mathematics that studies the properties of codes, in particular it focuses on the **error-detection** and **error-correction** capabilities of codes.

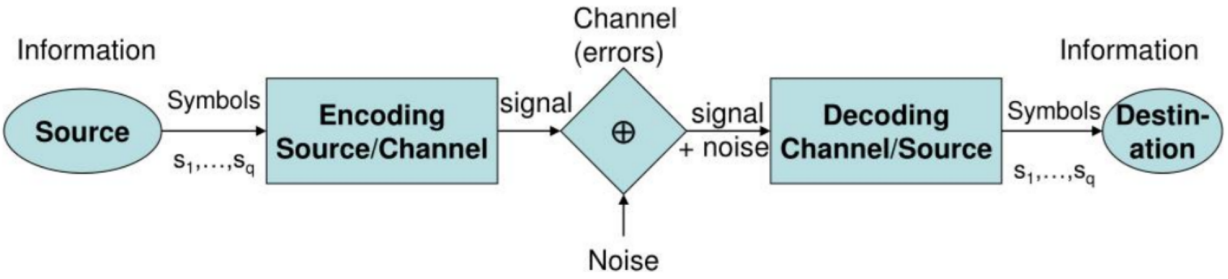


Figure 5: Coding theory overview

Each step shown in the figure above is crucial for the design and analysis of codes. The **encoding** step involves transforming the original message into a codeword using a specific encoding scheme. The **transmission** step is where the codeword is sent through a communication channel, which may introduce errors. The **decoding** step is where the received codeword is processed to recover the original message, and this is where error-detection and error-correction techniques come into play.
In the coding theory there are:

- F: finite alphabet, which is the set of symbols used to construct codewords. For example, in binary codes, the alphabet is $\{0, 1\}$.
- $m \in F^k$: information word or message, which is the original data that we want to encode. It consists of k symbols from the alphabet F.
- $c \in F^n$: codeword or input word, which is the encoded version of the message. It consists of n symbols from the alphabet F, where $n \geq k$.
- $y \in F^n$: received word or output word, which is the codeword that has been transmitted through the channel and may contain errors. It also consists of n symbols from the alphabet F.
- $\hat{c} \in F^n$ decoded word
- $\hat{m} \in F^k$ decoded message

4.1 Codewords

For our purposes, we will consider **additive channels**, i.e., given c, y $\in F^n$, then

$$y = c + e$$

where e is the error vector or error word.

We define a (n, M)-code C as non empty subset of F^n such that $|C| = n$ where n is the **length** of the codewords and M is the number of codewords in the code (**size**). *(In this case the notation |C| doesn't mean "number of elemnts of C", but it means the leght of th vector)*
The quantity $k \coloneqq \log_{|F|} M$ is called the **dimension** or information length of the code and $R \coloneqq \frac{k}{n}$ is called the **rate** of the code.
The elements of C are called **codewords**.

4.1.1 Distances

21: Hamming Distance

Given x, y $\in F^n$, the Hamming distance between x and y is defined as the number of positions (coordinates) in which x and y differ.
It is denoted as $d(x, y)$

22: Hamming Weight

Given x $\in F^n$, the Hamming weight of x is defined as the number of non-zero coordinates of x.
It is denoted as $w(x)$

23: Minimum Distance

Given a (n, M)-code C, the minimum distance of C is:

$$d \coloneqq min\{d(c_1, c_2), \forall c_1, c_2 \in C, c_1 \neq c_2\}$$

Note that $d(c_1, c_2)$ is the Hamming distance between c_1 and c_2.

The concept of minimum distance is crucial in coding theory, as it determines the error-detection and error-correction capabilities of the code. A code with a larger minimum distance can detect and correct more errors. Specifically, a

code with minimum distance d can detect up to $d-1$ errors and can correct up to $\lfloor \frac{d-1}{2} \rfloor$ errors (we will discuss the theorem later). (vedi Teorema 4.1.2).

Examples:
Let us consider $F = \{0,1\} = \mathbb{Z}_2 = \mathbb{F}_2$, we define the following code of leght $n = 3$ and size $M = 8$:

$$C = \{000, 001, 010, 011, 100, 101, 110, 111\}$$

In this case, the dimension is $k = \log_2 8 = 3$, this means that all the bites are "information bits".
The minimum distance is $d = 1$. In this case we are not able to detect and correct any error.

Let us consider $F = \{0,1\} = \mathbb{Z}_2 = \mathbb{F}_2$, we define the following code of leght $n = 6$ and size $M = 8$:

$$C = \{000000, 001111, 110011, 111100, 101010, 010101, 011001, 100110\}$$

In this case, the dimension is $k = \log_2 8 = 3$ and the rate is $R = \frac{3}{6} = \frac{1}{2} = 0.5$.
This means that we have 3 information bits and 3 redundancy bits.
The minimum distance is $d = 2$. In this case we are able to detect 1 error but we are not able to correct any error.

Let us consider $F = \{0,1\} = \mathbb{Z}_2 = \mathbb{F}_2$, we define the following code of leght $n = 9$ and size $M = 8$:

$$C = \{000000000, 111111111, 000111111, 111000000, 001001001, 110110110, 010010010, 101101101\}$$

In this case, the dimension is $k = \log_2 8 = 3$ and the rate is $R = \frac{3}{9} = \frac{1}{3} \approx 0.33$.
This means that we have 3 information bits and 6 redundancy bits.
The minimum distance is $d = 3$. In this case we are able to detect 2 errors and we are able to correct 1 error.

4.1.2 Conditions for correction and detection

24: Sphere of radius r

Given $x \in F^n$ and $r \in \mathbb{N}$, the sphere of radius r and center x is defined as:

$$S(x, r) \coloneqq \{x' \in F^n : d(x, x') \leq r\}$$

In an informal way, at the center of the sphere we have a codeword c, around c we have all the possible words that we can obtain by introducing at most r errors in c.
So, all that we can recieve by introducing at most r errors starting from c.

4

- A code C detects h_1 errors if and only if $\forall c \in C$, we have $S_{h_1}(c) \cap C = \{c\}$
- A code C corrects h_2 errors if and only if $\forall c_1, c_2 \in C$, we have $S_{h_2}(c_1) \cap S_{h_2}(c_2) = \emptyset$

First proposition, underline that in each sphere of radius h_1 and center c there is only c itself, so just one codeword.
Second proposition, underline that two different codewords c_1 and c_2 have two spheres of radius h_2 that do not intersect, so they are disjoint.

7: Error Detection and Correction Capabilities

Given a code C with minimum distance d, then C detects up to $d-1$ errors and corrects up to $\lfloor \frac{d-1}{2} \rfloor$ errors.

4.2 Linear codes

This sub-section is about "how contruct codes". In particular, we will focus on **linear codes**, which are a special class of codes that have a linear structure. We will always consider $F = \mathbb{F}_q$ a finite fiels with q elements, where q is a prime or a power of a prime.

25: Linear Code

A code C with parameters (n, M, d) is called linear if C is a linear subspace of F_q^n over F_q.

For linear codes, we use the notation [n, k, d] to denote a linear code of length n, dimension k and minimum distance d.
The difference (n - k) is called the **redundancy** of the code, and it represents the number of redundancy bits added to the information bits to form the

codewords.

Remark: In a linear code, the linear combination of two codewords is still a codeword.

Example: Let us construct a linear code of dimension k = 2 over $\mathbb{F}_2^5$.
It is sufficient to take two linearly indipendet vectors in $\mathbb{F}_2^5$, for example:

$$c_1 = (1, 0, 1, 1, 1)$$
$$c_2 = (1, 1, 1, 1, 0)$$

and thus all the codewords are $a_1c_1 + a_2c_2 \forall a_1, a_2 \in \mathbb{F}_2$, so we have:

$$C = \{00000, 10111, 11110, 01001\}$$

We can observe that the minimum distance is $d = 2$

Remark: For linear code C, we can observe that the null vector $0 = (000...0)$ is always a codeword and moreover

$$d(c_1, c_2) = d(c_1 - c_2, 0)$$

for all $c_1, c_2 \in C$, and thus

$$d = \min_{\substack{x \in C \\ x \neq 0}} w(x)$$

In general, if M is the size of a code, then, in order to evaluete d, we neeed to evaluate the Hamming distance between all the pairs of elements of C, i.e., $\binom{M}{2} \simeq M^2$.
For a linear code, it is sufficient to perform M-1 cheks.

4.2.1 Generator matrix

In general a matrix is a rectangular array of numbers, symbols, or expressions, arranged in rows and columns. In coding theory, we use matrices to represent linear codes, to perform encoding and decoding operations or to memorize the codewords of a linear code. For our purposes, we discuss a lot about the

generator matrix.

26: Generator Matrix

Given a linear code C with parameters [n, k, d], a generator matrix for C is a k x n matrix G whose rows form a basis for C.

All the codewords can be obtained using the generator matrix with all the information words, which are all the elements of $\mathbb{F}_q^k$, i.e.,

$$C = \{mG : m \in \mathbb{F}_q^k\}$$

Thus, an information word m is encoded into a codeword $c = mG$.
A linear code is completely determined by G, i.e., kn elements of F_q are sufficient to fully describe a code C that contains q^k vectors of length n (i.e., $q^k n$ elements of F_q).

A $k \times n$ generator matrix G is called **systematic** if ot has the form

$$G = (I|A)$$

where I is the $k \times k$ identity matrix and A is a $k \times (n-k)$ matrix.
A linear conde allows a systematic generator matrix if and only if the first k columns of any generator matrix are linearly independent.
If G is a systematic generator matrix, we can observe that given an information word $m = (m_1, m_2, ..., m_k) \in \mathbb{F}_q^k$ an information word, then

$$c = mG = (m_1, m_2, ..., m_k | p_1, p_2, ..., p_{n-k}) \in \mathbb{F}_q^n$$

i.e., the first k elements of a codeword are the elements of the information word.
This means that it is immediate to recover the information word from the codeword.

We define the **dot product** in $\mathbb{F}_q^n$ as follows:

$$x \cdot y \coloneqq x_1 y_1 + x_2 y_2 + ... + x_n y_n$$

for all $x, y \in \mathbb{F}_q^n$

27: Dual Code

Given a linear code $C \in \mathbb{F}_q^n$, the dual code of C is defined as:

$$C^\perp \coloneqq \{x \in \mathbb{F}_q^n : x \cdot y = 0, \forall y \in C\}$$

The generator matrix H of $C^\perp$ is called the **parity-check matrix** of C.

5

Given a recieved word $y \in \mathbb{F}_q^n$, we have

$$y \in C \iff yH^T = 0$$

6

If $G = (I_k|A)$, then $H = (-A^T|I_{n-k})$

Example: Let us consider linear code C with generator matrix

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Thus C is a linear code of length $n = 7$, dimension $k = 4$.
We can observe that

$$C = \{(m_1, m_2, m_3, m_4, m_1+m_3+m_4, m_1+m_2+m_4, m_1+m_2+m_3) : m_i \in \mathbb{F}_q\}$$

and

$$H = \begin{pmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}$$

from which we obtain that $y = (y_1, y_2, ..., y_7) \in C$ if and only if

$$\begin{cases} y_1 + y_3 + y_4 + y_5 = 0 \\ y_1 + y_2 + y_4 + y_6 = 0 \\ y_1 + y_2 + y_3 + y_7 = 0 \end{cases}$$

4.2.2 Identification of errors

As already said, one of the main goals of coding theory is to identify and correct errors.
To do this, we have to define the **nearest-codeword decoding**. It consists in: given a received word $y \in \mathbb{F}_q^n$, find a codeword $c \in C$ that minimizes the Hamming distance $d(y, c)$ or equivalently find an error $e \in \mathbb{F}_q^n$ of minimum Hamming weight s.t. $y - e \in C$.

Let C be a linear [n, k, d]-code and let H be a parity-check matrix for C. We define the function $s : \mathbb{F}_q^n \to \mathbb{F}_q^{n-k}$ such that

$$s(y) = yH^T$$

The function s is called the **syndrome** of y.

Given a recieved word $y = c + e$, we have that $s(y) = s(e)$.
Thus, given a recieved word y, the idea to detect and correct errors is the following:

1: Pre-compute the syndrome of all possible errors that the linear code is able to correct
2: evaluate the syndrom $s(y)$
3: if $s(y) = 0$, then there are no errors (we recieved a codeword)
4: if $s(y) \neq 0$, then we have an error, and we can identify the error by looking at the pre-computed syndromes; if $s(y) = s(e)$ for some error e, then correcr y with $y - e$

Example: Let C be a linear [4, 3]-code over $\mathbb{F}_3 = \mathbb{Z}_3 = \{0, 1, 2\}$ with parity-check matrix

$$H = \begin{pmatrix} 1 & 1 & 0 & 2 \\ 0 & 2 & 1 & 2 \end{pmatrix}$$

Thus, the linear code is defined by the following equations:

$$\begin{cases} x_1 = -x_2 + x_4 \\ x_3 = x_2 + x_4 \end{cases}$$

and then

$$C = \{(a + b, -a, -a + b, b) : a, b \in \mathbb{Z}_3\}$$

We can check that the minimum distance is $d = 3$, hence we are albe ro detect 2 errores and to correct only 1.

The possible errors that we are able to correct are

$$e_1 = (1, 0, 0, 0),\ e_2 = (0, 1, 0, 0),\ e_3 = (0, 0, 1, 0),\ e_4 = (0, 0, 0, 1),$$
$$2e_1 = (2, 0, 0, 0),\ 2e_2 = (0, 2, 0, 0),\ 2e_3 = (0, 0, 2, 0),\ 2e_4 = (0, 0, 0, 2)$$

Thus, if we receive the word $y = (2, 1, 2, 1)$, we evaluate the syndrome

$$s(y) = yH^T = (2,1,2,1)\begin{pmatrix}1 & 0\\ 1 & 2\\ 0 & 1\\ 2 & 2\end{pmatrix} = (2,0)$$

since is different from 0, we check the pre-computed syndromes of the possible errors and we find that it is equal to $s(2e1)$

4.2.3 Goppa Codes

A (binary) Goppa code is a linear code defined by menas of a particular parity check matrix (and consequently a generator matrix) with a fast method for 'decoding', i.e., for correctiong errors.
Given $\alpha_1, \alpha_2, ..., \alpha_n \in \mathbb{F}_{2^m}$ and $g(x) \in \mathbb{F}_{2^m}[x]$ irreducible of degree t. Define a linear code C with parity-check matrix:

$$H = \begin{pmatrix} 1 & 1 & ... & 1 & \\ \alpha_1 & \alpha_2 & \alpha_3 & ... & \alpha_n \\ \alpha_1^2 & \alpha_2^2 & \alpha_3^2 & ... & \alpha_n^2 \\ ... & ... & ... & ... & ... \\ \alpha_1^{t-1} & \alpha_2^{t-1} & \alpha_3^{t-1} & ... & \alpha_n^{t-1} \end{pmatrix} \cdot D$$

where D is a diagonal matrix where in the diagonal there are the elements $\frac{1}{g(\alpha_i)}$ and t is the dedree of $g(x)$

if $g(x)$ is an irreducible polynomial, then the minimum distance of the Goppa code is at least $2t + 1$. Moreover, the syndrome of a word y has a very nice closed form:

$$s(y) = (s_0, s_1, ..., s_{t-1})$$

where $s_0, s_1, ..., s_{t-1}$ are the coefficients of the polynomial

$$\textstyle\sum_{i=1}^{n} \frac{y_i}{x-\alpha_i} \mod g(x).$$

For performing the correction of errors occured in a received word $y = c + e$, its **error locating polynomial is used**

$$\sigma(x) = \textstyle\prod_{i:e_i \neq 0}(x - \alpha_i)$$

where the product ranges over the indexs i such that $e_i \neq 0$.
Thus, the main problem is to evaluate the polynomial $\sigma(x)$, associated to a given received word.
For this reason, we can use the **Patterson algorithm**, which is an efficient algorithm for finding the error locating polynomial $\sigma(x)$ given the syndrome $s(y)$.

Algorithm 4 Patterson Algorithm

1: Evaluate the syndrome $s(y)$ and denote by $s(X)$ the corresponding polynomial
2: Find $h(X)$ s.t. $s(X)h(X) \equiv 0 \mod g(X)$; if $h(X) = 0$, then output $\sigma(X) = 1$
3: Evaluate $d(X)$ s.t. $d(X)^2 \equiv h(X) + X \mod g(X)$
4: Find $a(X)$ and $b(X)$ s.t. $d(X)b(X) \equiv a(X) \mod g(X)$
5: Set $\sigma(X) = a(X)^2 + Xb(X)^2X$
6: Use $\sigma(X)$ to find the error locations

Summary

Let C be a (n,M,d)-code, able to detect d-1 errors and to correct $t = [(d-1)/2]$ errors. Let y be a received word.

- If C is not structured, for detecting the errors, we need to compare y with all the codewords and, if $y \notin C$, we can correct the errors with the idea of the nearest-codeword decoding.
- If C is a linear code, for detecting the errors, it is sufficient to evaluate the syndrome and, if it is not zero, we can correct the errors comparing this value with the pre-computed syndromes of all possible errors
- If C is a Goppa code, we detect the errors as a classical linear code and, if there are errors, we correct them with the Patterson algorithm, which is more efficient than the syndorme method when we need to correct many errors.

5 Code-base cryptosystems

As already mentioned, code-based cryptography is based on the hardness relying on coding theory. In particular, the security of code-based cryptosystems relies on the difficulty of correcting errors in a linear code.

Thus, one idea is to make public a 'random' code and keep private an equivalent 'structured' code, where it is fast to correct the errors. Then, the plaintext could be an information word and the ciphertext could be the corresponding codeword plus an error (with high Hamming weight), so that only who has the 'structured' code is able to recover the plaintext (i.e., to recover the codeword and consequently the corresponding information word). In some schemes, the plaintext could also be the codeword or the error vector.

The first cryptosystem based on coding theory was proposed by Robert McEliece in 1978

5.1 McEliece Cryptosystem

The McEliece cryptosystem is based on the hardness of decoding a random linear code. The original proposal uses Goppa codes, which are a class of error-correcting codes that have efficient decoding algorithms. The security of the McEliece cryptosystem relies on the difficulty of decoding a random linear code, which is believed to be hard even for quantum computers.

5.1.1 Key Generation

1: Choose integers, m, t and fix $n = 2^m$
2: Select a binary Goppa code $\mathcal{C}$ of length n, able to correct t errors (this is equivalent to select an irreducible polynomial of degree t over $\mathbb{F}_{2^m}$)
3: Let G be the generator matrix $k \times n$ of $\mathcal{C}$, where k is the dimension of the code (observe that $k \geq n - mt$)
4: Choose a random $k \times k$ invertible matrix S
5: Choose a random $n \times n$ permutation matrix P
6: Compute $\hat{G} = SGP$

The pair $\hat{G}$ is the public key, while the private key is the triple S, G, P.

5.1.2 Encryption

Vector y is the ciphertext.

1: Let m be the plaintext, where m is a binary vector of length k (so $m \in \mathbb{F}_2^k$)
2: Choose a random error vector e of length n and Hamming weight t (i.e., e has exactly t non-zero entries)
3: Compute the vector $y = m\hat{G} + e$

5.1.3 Decryption

Let $y \in \mathbb{F}_2^n$ be the ciphertext. Remember that the private key is the triple S, G, P. The decryption process is as follows:

1: Compute $y' = yP^{-1}$
2: Use Patterson's algorithm to decode y'.
3: Let c be the decoded codeword and m' be the corresponding information word
4: Evaluate $m'S^{-1}$

The output of the decryption process is $m'S^{-1}$, which is the original plaintext m.

Proof of correctness

Let us verify that the decryption process:

$$\begin{aligned} y = cP^{-1} &= (m\hat{G} + e)P^{-1} = mSGPP^{-1} + eP^{-1} = (mS)G + eP^{-1} \\ &= m'G + f \end{aligned}$$

For semplicity we have denoted $f = eP^{-1}$ (that is a vector in $\mathbb{F}_2^n$ with Hamming weight t) and $m' = mS$ (that is the vector in $\mathbb{F}_2^n$

Let us observe that m' is the information word, and $m'G$ is the codeword of the Goppa code.
Moreover, f is an error vector with Hamming weight t, because e is a vector in $\mathbb{F}_2^n$ with Hamming weight t and its weight is not modified by P^{-1}, since P^{-1} is a permutation matrix and consquently it only permutes the entries of e.

Thus, $y \in \mathbb{F}_2^n$ is a recieved word obteined from a codeword of the Goppa code $m'G$ plus an error vector f; t errors have been added to the codewor $m'G$.
Thus, applying the Patterson's algorithm to y we are able to remove the errors and to recover the codeword $m'G$

$$\text{Patterson}(y) = \text{Patterson}(m'G + f) = m'G = y'$$

From $y' \in C_{Goppa}$ we can recover the information word which is m' (remember that m' is (mS))

Finally, we can compute $m'S^{-1} = mSGS^{-1} = mG$, and since G is the generator matrix of the Goppa code, we have that m is the original plaintext.

5.1.4 Security of McEliece

> ***Authr's note:*** *For the following discussion, I will refer to the original paper of McEliece, so (according to the lesson) I will use the oringinal notation of the paper*

As discussed by McEliece in his original paper, we evaluate the security checking the following points:

- COA (security against a ciphertext-only attack)
- Brute-force attack

COA

Let us suppose to know a ciphertext $\underline{x} = (x_1, x_2, \ldots, x_n) \in \mathbb{F}_2^n$ obteined from the McEliece cryptosystem.
Thus we know that $\underline{x} = \underline{u}G' + \underline{z}$, where:

- $\underline{u} = (u_1, u_2, \ldots, u_k) \in \mathbb{F}_2^k$ is the plaintext
- $G' \in \mathbb{F}_2^{k \times n}$ is the public key
- $\underline{z} = (z_1, z_2, \ldots, z_n) \in \mathbb{F}_2^n$ is a random UNKNOWN vector with Hamming weight t

We would like to recover the plaintext $\underline{u}$
From $\underline{x} = \underline{u}G' + \underline{z}$, we have:

$$(x_1, x_2, \ldots, x_n) = (u_1, u_2, \ldots, u_k)G' + (z_1, z_2, \ldots, z_n)$$

but we know only the vector $\underline{x}$ (green), others are unknown (red).

Thus, we have a linear sysetm with n equations and $k + n$ unknowns:

$$\circledast \begin{cases} x_1 = u_1 g'_{11} + u_2 g'_{12} + \ldots + u_k g'_{k1} + z_1 \\ x_2 = u_1 g'_{12} + u_2 g'_{22} + \ldots + u_k g'_{k2} + z_2 \\ \vdots \\ x_n = u_1 g'_{1n} + u_2 g'_{2n} + \ldots + u_k g'_{kn} + z_n \end{cases}$$

Assuming that the equations are all linearly independent, we have that this linear system has k degrees of freedom, so k free variables(by Rouché-Capelli theorem)
Since our variables take values in $\mathbb{F}_2$, this means that we have 2^k possible solutions and for finding the plaintext we should check all these 2^k possible solutions.
It is simple to see that the complexity of this attack is $O(2^k)$, so if $k = 524$ (as in the original paper of McEliece) we have a security of 524 bits.

This attack can be easily improved.
Indeed in $\circledast$we are only interested into the k unknowns $u_1, u_2, \ldots, u_k$. Thus if we are able to select k equations among the n equations in $\circledast$where the z_i's are all zero, then we have a linear system with k equations and k unknowns which has one solution (which is the plaintext).

However, the probability of selecting such equations is

$$(1 - \tfrac{t}{n})^k$$

Indeed, the probability that $z_i \neq 0$ is $\frac{t}{n}$ (since $\underline{z}$ is a vector of length n and Hamming weight t), so the probability that $z_i = 0$ is $1 - \frac{t}{n}$.

Thus, in average, we need to randomly select $(1 - \frac{t}{n})^{-k}$ linear systems (with k equations) in order to obtain one linear system where all the z_i's are zero. Thus, the total cost is

$$k^3(1 - \tfrac{t}{n})^{-k} \sim 2^{65}$$

Brute force attacks

Let us consider the parameters of the original paper of McEliece, that is $m = 10, t = 50, n = 2^{10} = 1024$ and $k \geq n - mt = 1024 - 10 \cdot 50 = 524$.
Let us check that brute force is infeasible.

Recall that the private key is the triple S, G, P with $S \in \mathbb{F}_2^{k \times k}$ invertible matrix, $G \in \mathbb{F}_2^{k \times n}$ generator matrix of the Goppa code and $P \in \mathbb{F}_2^{n \times n}$ permutation matrix.
Check the number of possible private keys by checking the number of possible S, G, P:

- The number of possible matrices S:

It is the number of matrices in $\mathbb{F}_2^{k \times k}$ which are invertible (determinant is not zero) or equivalently the number of matrices in $\mathbb{F}_2^{k \times k}$ which have linearly independent columns.

In general, the number of matrices in $\mathbb{F}_2^{k \times k}$ with linearly independent columns is

$$q^k - 1 \text{ (for the first column)} \cdot q^k - q \text{ (for the second column)} \cdot q^k - q^2 \text{ (for the third column)} \cdot \ldots \cdot q^k - q^{k-1} \text{ (for the last column)}$$

Indeed, the first column $\underline{c}_1$ must be a non-zero element of $\mathbb{F}_2^k$ (non-zero vector of length k).
Observing that $|\mathbb{F}_q^k| = q^k$, we have that the number of possible choices for $\underline{c}_1$ is $q^k - 1$.

The second column $\underline{c}_2$ must be linearly independent from $\underline{c}_1$.
Thus, we must have $\underline{c}_2 \neq a\underline{c}_1 \forall a \in \mathbb{F}_q$ and thus we have $q^k - q$ possible choices for $\underline{c}_2$.

The third column $\underline{c}_3$ must be linearly independent from $\underline{c}_1$ and $\underline{c}_2$.
Thus, we must have $\underline{c}_3 \neq a_1\underline{c}_1 + a_2\underline{c}_2 \forall a_1, a_2 \in \mathbb{F}_q$ and thus we have $q^k - q^2$ possible choices for $\underline{c}_3$.

Continuing in this way, we have that the last column $\underline{c}_k$ must be linearly independent from $\underline{c}_1, \underline{c}_2, \ldots, \underline{c}_{k-1}$.
Thus, we must have $\underline{c}_k \neq a_1\underline{c}_1 + a_2\underline{c}_2 + \ldots + a_{k-1}\underline{c}_{k-1} \forall a_1, a_2, \ldots, a_{k-1} \in \mathbb{F}_q$ and thus we have $q^k - q^{k-1}$ possible choices for $\underline{c}_k$.

Thus, the total number of possible S is

$$(2^k - 1)(2^k - 2)(2^k - 2^2) \cdot \ldots \cdot (2^k - 2^{k-1}) > 2^k = 2^{256}$$

- The number of possible matrices P:

It is equivalent to the number of possible permutations of n elements, that is $n!$.

$$n! \geq (\tfrac{n}{2})^{\frac{n}{2}} = (2^9)^{2^9} = 2^{9 \cdot 512}$$

- The number of possible matrices G:

It is equivalent to the number of binary Goppa codes able to correct t errors, that is the number of irreducible polynomials of degree t over $\mathbb{F}_{2^m}$

In general, the number of irreducible polynomials of degree t over $\mathbb{F}_{2^m}$ is

$$\tfrac{1}{t} \sum_{d|t} \mu(d) q^{\frac{t}{d}}$$

where

$$\mu(d) \begin{cases} 1 & \text{if } d = 1 \\ 0 & \text{if } d \text{ not squarefree} \\ (-1)^a & \text{if } d \text{ square free in the number of its prime factors} \end{cases}$$

In our case, we have

$$\tfrac{1}{t} \sum_{d|t} \mu(d)(2^m)^{\frac{t}{d}} > \tfrac{1}{t}(2^m)^t = \tfrac{1}{50} 2^{10 \cdot 50} = \tfrac{1}{50} 2^{500}$$

5.1.5 McEliece trapdoor "improvements"

A trapdoor function is a function that is easy to compute in one direction, but hard to invert unless you have some secret information (the trapdoor).
In the McEliece cryptosystem, the trapdoor is

$$G' = SGP$$

where G' is the public key, while S, G, P are the private key.

In ordere to save space, someone could try to use trapdoor only

$$G' = GP \text{ or } G' = SG$$

However, these "improvements" are not secure.

Indeed, if we only know G', it is not possible to recover the private key (S, G) or (G, P).
Moreover, the proof of correctness (with the some modifications) still works also in these cases.
However, if we consider $G' = SG$ we are not hiding Goppa code with the generator matrix G, because SG only provides a change of basis. This means that the linear code with generator matrix G' is the same as the linear code with generator matrix G (Goppa code).
In other words, G and $G' = SG$ are two generator matrices of the same linear code (remeber that a generator matrix has a rows a basis of the linear code).

If we consider $G' = GP$, then the linear code C' with generator matrix G' is equivalent but not equal to the linear code C with generator matrix G.
However, the exist some method in order to recover G from G' when, for example, C is a Reed-Solomon code (that is a particular class of linear code).

Thus, we don't consider these "improvements" as secure, because they don't hide the structure of the Goppa code, and consequently they are vulnerable to structural attacks (that is attacks that try to recover the private key from the public key by exploiting the structure of the code).

5.1.6 Digital Signature (McEliece)

Usaually, when we have a public key cryptosystem, we can also use it to build a digital signature scheme exploiting the idea of signing a message by encrypting it with the private key and verifying the signature by decrypting it with the public key.

In the case of McEliece, following the same idea, we could:

1. Apply the decrytion algorithm to a vector $y \in \mathbb{F}_2^n$ and find $m \in \mathbb{F}_2^k$ and $e \in \mathbb{F}_2^n$ such that $y = mG' + e$ and send (m, e) as signature of y.

2. To verify the signature, we can check that $y = mG' + e$ and accept the signature if and only if is true.

However, this scheme has a problem because we are not able to sign all the possible messages $y \in \mathbb{F}_2^n$, but only the messages $y \in \mathbb{F}_2^n$ obtained from a codeword of C' and adding an error of Hamming weight t.

The number of these kind of messages is

$$2^k \cdot \sum_{i=0}^{t} \binom{n}{i}$$

Indeed, if we fix a codeword, we have

- n possible positions to insert 1 error
- $\binom{n}{2}$ possible positions to insert 2 errors
- $\vdots$
- $\binom{n}{t}$ possible positions to insert t errors

The number of all elements of $\mathbb{F}_2^n$ is 2^n. Thus, using McEliece parameters $(n = 2^{10}, t = 50, k = 254)$, we have that the number of messages in $\mathbb{F}_2^n$ that we can sign is

$$\frac{2^k \sum_{i=0}^{t} \binom{n}{i}}{2^n} \sim \frac{2^{808}}{2^{1024}} \sim \frac{1}{2^{216}}$$

VERY LOW!
So this scheme is not secure, because an attacker can easily forge a signature by randomly selecting a message $y \in \mathbb{F}_2^n$ and checking if it is a valid signature.

5.2 Niederreiter Cryptosystem

The Niederreiter cryptosystem is a variant of the McEliece cryptosystem.

Let us recall that in the McEliece cryptosystem, the ciphertext is

$$c = mG' + e \in \mathbb{F}_2^n$$

where $m \in \mathbb{F}_2^k$ is the plaintext, $G' \in \mathbb{F}_2^{k \times n}$ is the public key and $e \in \mathbb{F}_2^n$ is a random error vector with Hamming weight t.

Let us observe that, if we find $\underline{e}$ we are able to recover $\underline{m}$ from $\underline{c}$.

However, the number of possible error vector $\underline{e}$ is very large. Indeed, the number of possible error vector $\underline{e} \in \mathbb{F}_2^n$ with Hamming weight t is

$$\binom{n}{t} = \frac{n \cdot (n-1) \cdot \ldots \cdot (n-(t-1))}{t!} \geq \frac{(\frac{n}{t})^t}{t^t} = \left(\frac{n}{2t}\right)^t$$
$$=_{\text{consider the McEliece parameters}} \left(\frac{2^{10}}{2^3}\right)^{50} \geq \left(\frac{2^{10}}{2^3}\right)^{50} = 2^{150}$$

Let us observe that, from $\underline{c} = \underline{m}G' + \underline{e}$, we know the syndrome of $\underline{c}$, because is $H'\underline{c} = H'\underline{e}$, where H' is the parity check matrix of C', but in a random linear code C', if we only know the syndrome of $\underline{c}$, we are not able to recover $\underline{e}$ (syndrome decoding problem). Thus, the idea of Niederreiter is to hide a plaintex in $H'\underline{e}$ where $\underline{e}$ become the plaintext.

This, essentially, is the idea of the Niederreiter cryptosystem, that is a variant of the McEliece cryptosystem where the plaintext is the error vector $\underline{e}$ and the ciphertext is the syndrome $H'\underline{e}$.

5.2.1 Key Generation

1: Choose integers, m, t and fix $n = 2^m$
2: Select a binary Goppa code $\mathcal{C}$ of length n, able to correct t errors (this is equivalent to select an irreducible polynomial of degree t over $\mathbb{F}_{2^m}$)
3: Let $H \in \mathbb{F}_2^{(n-k)\times(n-k)}$ be the parity check matrix
4: Select an invertible matrix $S \in \mathbb{F}_2^{(n-k)\times(n-k)}$ and a permutation matrix $P \in \mathbb{F}_2^{n \times n}$
5: Compute $H' = SHP$, a parity check matrix of a random linear code

The H' is the public key, while the private key is the triple S, H, P.

5.2.2 Encryption

Let $\underline{m} \in \mathbb{F}_2^n$ with Hamming weight t be the plaintext The vector $\underline{c}$ is the

1: Compute the syndrome $\underline{c} = H'\underline{m}$

ciphertext.

5.2.3 Decryption

Let $c \in \mathbb{F}_2^{n-k}$ be the ciphertext. Remember that the private key is the triple S, H, P. The decryption process is as follows:

1: Compute $\underline{s} = S^{-1}\underline{c}$ (is the syndrome of error in the Goppa code)
2: Compute the error vector $\underline{e}$ corresponding to the syndrome $\underline{s}$ using Patterson's algorithm
3: Compute $P^{-1}\underline{e} = m \longleftarrow$the plaintext

---

5.3 Comparison

It is known that PQC has a size limitation, that is the public key and the ciphertext of a PQC scheme are usually larger than the public key and the ciphertext of a classical cryptosystem (like RSA or ECC).

To better understand this, let us compare the size of the public key and the ciphertext of McEliece and Niederreiter cryptosystem with the size of the public key and the ciphertext of RSA

5.3.1 RSA (basic)

Consider the monulo $N \sim 2048$ bits.
This ensure a bit-secure > 100 bits, so it's menas that cost of the attack is $O(2^{100})$.

Plaintext

$m \in \mathbb{Z}_n$, thus it has a size of 2048 bits.

Ciphertext

$c \in \mathbb{Z}_n$, thus it has a size of 2048 bits.

Ratio

$\frac{2048}{2048} = 1$

Public-key

(e, N) where $e \in \mathbb{Z}_{\phi(N)}$.
However, e is usually 65537, because it has a convenient bit-expression. $655037 = 100...001$ and exponentiation is more convenient if the exponent has this kind of binary representation.
This because we use the square-and-multiply algorithm for computing powers.

Algorithm 5 Square-and-multiply

```
1: Input: x, n
2: Let n_1, n_2, ..., n_k be the binary representation of n
3: Set r = x
4: for i = 2 to k do
5:     if n_i = 0 then
6:         r = r · r
7:     else
8:         r = r · r AND r = r · x
9:     end if
10: end for
```

Thus, the public key has a size of 2048 bits.

Secret-key

$d \in \mathbb{Z}_{\phi(N)}$
Since $\phi(N) \sim N$ we have that d has a size of 2048 bits.

5.3.2 McEliece (basic)

Consider the parameters of the original paper of McEliece, that is $m = 10, t = 50, n = 2^{10} = 1024$ and $k = n - mt = 524$.

Thus paramenters ensure 65 bits of security, that is the cost of the attack is $O(2^{65})$.

Plaintext

$m \in \mathbb{F}_2^k$, thus it has a size of 524 bits.

Ciphertext

$c \in \mathbb{F}_2^n$, thus it has a size of 1024 bits.

Ratio

$\frac{1024}{524} \sim 2$

Public-key

$G' \in \mathbb{F}_2^{k \times n}$, thus it has a size of $k \cdot n = 524 \cdot 1024$ bits.

Secret-key

S, G, P, where $S \in \mathbb{F}_2^{k \times k}$, $G \in \mathbb{F}_2^{(k \times n}$, and $P \in \mathbb{F}_2^{n \times n}$ (there are n! permutations of n elements, so we can encode the permutation matrix P with $\log_2(n!)$ bits).
Thus, the secret key has a size of $524 \cdot 524 + 524 \cdot 1024 + 8749$ bits.

5.3.3 Niederreiter (basic)

Consider the parameters of the original paper of McEliece

Plaintext

$m \in \mathbb{F}_2^n$ with Hamming weight t. Since there are $\binom{n}{t}$ vectors of this kind, then we can describe them with $\log_2(\binom{n}{t})$ bits.
Thus, the plaintext has a size of $\log_2(\binom{n}{t}) \sim 284$ bits.

Ciphertext

$c \in \mathbb{F}_2^{n-k}$, thus it has a size of 500 bits.

Ratio

$\frac{500}{284} = 1.76$

Public-key

$H' \in \mathbb{F}_2^{(n-k) \times n}$, thus it has a size of $(n-k) \cdot n = 500 \cdot 1024$ bits.

Secret-key

(S, H, P), where $S \in \mathbb{F}_2^{(n-k) \times (n-k)}$, $H \in \mathbb{F}_2^{(n-k) \times n}$, and $P \in \mathbb{F}_2^{n \times n}$.
Thus, the secret key has a size of $500 \cdot 500 + 500 \cdot 1024 + 8749$ bits.

5.4 Fundamental Function in Classic McEliece

By analyzing the Classic McEliece presentation, we can see three fundamental functions:

- **MatGen** (key generation)
- **Encode** (encryption)
- **Decode** (decryption)

5.4.1 MatGen

MatGen($\ulcorner$) where $\ulcorner = (g, \alpha_0, \ldots, \alpha_{n-1})$ with g is an irreducible polynomial of degree t over $\mathbb{F}_q$ and $\alpha_0, \ldots, \alpha_{n-1}$ are n distinct elements of $\mathbb{F}_q$.
$\ulcorner$ rapresents a Goppa code over $\mathbb{F}_q = \mathbb{F}_{2^m}$

CASE $(\mu, \nu) = (0, 0)$

1: Compute $\tilde{H}$ parity check matrix of the Goppa code defined by $\ulcorner$; $\tilde{H} \in \mathbb{F}_q^{t \times n}$
2: Compute $\hat{H} \in \mathbb{F}_2^{mt \times n}$ parity check matrix of the Goppa code over $\mathbb{F}_2$(knowing $\hat{H}$ is not possible to recover $\ulcorner$)
3: Reduce $\hat{H} \leftarrow$ i.e. use Gaussian elimination in order to transform in reduced row echelon form
4: Return $(T, \ulcorner)$ if in 3) we obtain a systemetic form, else return $\perp$ (failure)

CASE $(\mu, \nu) \neq (0, 0)$

1: Compute $\tilde{H}$ parity check matrix of the Goppa code defined by $\ulcorner$; $\tilde{H} \in \mathbb{F}_q^{t \times n}$
2: Compute $\hat{H} \in \mathbb{F}_2^{mt \times n}$ parity check matrix of the Goppa code over $\mathbb{F}_2$(knowing $\hat{H}$ is not possible to recover $\ulcorner$)
3: Reduce $\hat{H} \leftarrow$ i.e. use Gaussian elimination in order to transform in reduced row echelon form
4: Return $\perp$ if in 3) we don't obtain a semi-systematic form, else perform steps 4), 5) and 6) (of paper)
5: Return $(T, c_{mt-\mu}, \ldots, c_{mt-1}, \ulcorner')$ where $\ulcorner' = (g, \alpha'_0, \ldots, \alpha'_{n-1})$ $((\alpha'_0, \ldots, \alpha'_{n-1})$ is a permutation of $(\alpha_0, \ldots, \alpha_{n-1})$ in order to avoid the permutation in de decoding phase)

5.4.2 Encode

Encode$(\underline{e}, T)$ where $\underline{e} \in \mathbb{F}_2^n$, $w(\underline{e}) = t$ and T is the output of MatGen($\ulcorner$).

1: Compute $H = (I|T)$
2: Compute $C = H \cdot \underline{e} \in \mathbb{F}_2^{mt}$ (remeber that $n - k = mt$ so $n = mt + k$)
3: Return C as ciphertext

5.4.3 Decode

Decode$(C, \ulcorner')$ where $C \in \mathbb{F}_2^{mt}$ output of Encode and $\ulcorner'$ is the last part of the output of MatGen.

1: $\underline{\nu} = (C, (0, 0, \ldots, 0)[k{-}times]) = (\frac{C}{\underline{0}}) \in \mathbb{F}_2^n$ (ν is a received word abtained from a codeword plus an error vector $\underline{e}$)
2: Compute Patterson$(\underline{\nu}) = \underline{c} \in \mathbb{F}_2^n$ codeword
3: $\underline{e} = \underline{\nu} + \underline{c}$
4: **if** $w(\underline{e}) = t$ and $C = H \cdot \underline{e}$ **then**
5: Return $\underline{e}$ as plaintext
6: **else**
7: Return $\perp$ (failure)
8: **end if**

Proof of correctness:

This algorithm is correct if it is true that $\underline{\nu}$ (constructed in step 1) is a received word obtained from a codeword plus an error vector $\underline{e} \iff \underline{nu}$ has the same syndrom as $\underline{e} \iff H\nu = H\underline{e}$.
Let us check that considering $\underline{\nu}$ as in 1) it is true that $H\nu = H\underline{e}$:

$$H\underline{\nu} = (I|T)(\frac{C}{\underline{0}}) = C = H \cdot \underline{e}$$

5.5 KEM comparison

To better understand the difficult related to the implementation of a KEM based on McEliece, let us compare the size of the public key and the ciphertext of a KEM based on McEliece with the size of the public key and the ciphertext of a KEM based on RSA.

5.5.1 Classic McEliece KEM

Session key

$K \in \mathbb{F}_2^{256}$ so we have 256 bits

Ciphertext

$c \in \mathbb{F}_2^{mt} = \mathbb{F}_2^{n-k}$, thus it has a size of mt bits

Public-key

$T \in \mathbb{F}_2^{(mt)\times k} = \mathbb{F}_2^{(n-k)\times k}$, thus it has a size of $mt \cdot k$ bits

Secret key

it is a quintuple $(\delta, c, g, \alpha, s)$ where:

- δ is a seed, 256 bits
- c is a column swapped, x bits
- g is an irreducible polynomial of degree t over $\mathbb{F}_q = \mathbb{F}_{2^m}$, thus it has a size of $t \cdot m$ bits
- α is a ordering of all elements of $\mathbb{F}_q = \mathbb{F}_{2^m}$, via Bens representation, thus it has a size of $(2m-1)2^{m-1}$ bits

- s is an ausiliary vector, $s \in \mathbb{F}_2^n$, thus it has a size of n bits

Thus, the secret key has a size of $256 + \mathrm{x} + t \cdot m + (2m-1)2^{m-1} + n$ bits

5.5.2 RSA KEM

Session key

$K \in \mathbb{F}_2^{256}$ so we have 256 bits

Ciphertext

$c \in \mathbb{Z}_N$, a bits where N is the modulus

Public-key

N, a bits

Secret key

It is a quintuple $(p, q, d_p, d_q, q_{inv})$ where:

- p, q are prime s.t. $N = pq$, thus they have a size of $\frac{a}{2}$ bits
- $d_p = d \mod \varphi(p)$, thus it has a size of $\frac{a}{2}$ bits
- $d_q = d \mod \varphi(q)$, thus it has a size of $\frac{a}{2}$ bits
- $q_{inv} = q^{-1} \mod p$, thus it has a size of $\frac{a}{2}$ bits

Thus, the secret key has a size of $5 \cdot \frac{a}{2}$ bits

It possible visualize the comparison, using real numbers.

For CM, we have $m = 12, t = 64, n = 3488, v = (0 \text{ or } 64)$, 0 if we use the semi-systematic form, 64 if we use the systematic form

For RSA, we choose N s.t. a = 3072 bits.

Metric	CM	RSA
Security	126 bits	128 bits
Session key	256 bits	256 bits
Ciphertext	$12 \cdot 64 = 7698$ bits ~ 0.1 KB	3072 bits ~ 0.4 KB
Public key	$12 \cdot 64 \cdot 2720 \sim 2.000.000$ bits ~ 250 KB	3072 bits ~ 0.4 KB
Secret key	$256 + 64 + 768 + 47104 + 3488 \sim 50.000$ bits ~ 6 KB	$5 \cdot \frac{3072}{2} \sim 7.680$ bits ~ 1 KB

In a more realistic scenario, we can consider the parameters of CM with $m = 13, t = 128, n = 8192, v = 64$ and the parameters of RSA with N s.t. a = 15360 bits.

Metric	CM	RSA
Security	256 bits	256 bits
Session key	256 bits	256 bits
Ciphertext	$13 \cdot 128 = 1664$ bits ~ 0.2 KB	15360 bits ~ 1.9 KB
Public key	$1664 \cdot 6528 \sim 10.000.000$ bits ~ 1250 KB	15360 bits ~ 1.9 KB
Secret key	$256 + 64 + 1664 + 1024000 + 8192 \sim 112.000$ bits ~ 14 KB	$5 \cdot \frac{15360}{2} \sim 38.400$ bits ~ 4.75 KB

5.6 Hamming Quasi-Cycling

> ***Authr's note:*** *For the HQC, I strongly suggest to read the original paper, because the professor only gives a very high level overview of the scheme, and it's not possible to understand the details of the scheme without reading the original paper.*

Before understend the high level details, let me give the pricipal definitions.

28: Cycling code

A cycling code $C \subseteq \mathbb{F}_q^n$ is cyclic $\iff$ $\forall \underline{c} \in C$ we have $\sigma(\underline{c}) \in C$ where $\sigma(\underline{c}) = \sigma(c_0, c_1, \ldots, c_{n-1}) = (c_{n-1}, c_0, c_1, \ldots, c_{n-2})$ is the cycling operator (so it rapresent a "special permutation").

Remember that we can always rapresent a vector of lenght n as a polynomial of degree at most n-1, i.e., an element of $\mathbb{F}_q[x]/f(x)$ where $f(x)$ is a polynomial of degree n NOT NECESSARILY irreducible.

Let us consider $R = \mathbb{F}_q[x]/(X^n - 1)$ then

$$\underline{c} = (c_0, c_1, \ldots, c_{n-1}) \in \mathbb{F}_q^n \text{ can be rapresented as } c(x) = c_0 + c_1 x + \ldots + c_{n-1}x^{n-1} \in R$$

Thus, we can observe that

$$\begin{aligned}\sigma(\underline{c}) &= (c_{n-1}, c_0, c_1, \ldots, c_{n-2}) \\ &\leftrightarrow c_{n-1} + c_0 x + c_1 x^2 + \ldots + c_{n-2}x^{n-1} \\ &= x \cdot c(x) \text{ remebering that } x^n = 1 \text{in R}\end{aligned}$$

Thus, we can say that a code $C \subseteq \mathbb{F}_q^n$ is cycling if and only if is closed under the action of σ.
Equivalently a code $C \subseteq R$ is cycling if and only if is closed under the action of x.
Moreover, we can also have that

$$c(x) \in C \implies x \cdot c(x) \in C \implies x^2 \cdot c(x) \in C \implies \ldots \implies x^h \cdot c(x) \in C \text{ for every } h \in \mathbb{N}$$

Moreover, since C is linearm it is closer under the sum:

$$c(x) \in C \implies x^{h_1} \cdot c(x) \in C \text{ and } x^{h_2} \cdot c(x) \in C \implies x^{h_1} \cdot c(x) + x^{h_2} \cdot c(x) \in C$$

and it is also closed under multiplication by a scalar

$$c(x) \in C \implies x^h \cdot c(x) \in C \text{ and } a_2 \cdot x^{h_2} \cdot c(x) \in C \implies a_1 \cdot x^{h_1} \cdot c(x) + a_2 \cdot x^{h_2} \cdot c(x) \in C$$

Thus, a cycling code C is closed under the multiplication by ANY element of R (since R is generated by x):

$$c(x) \in C \implies g(x) \cdot c(x) \in C, \forall g(x) \in R$$

5.6.1 HQC overview

Hamming Quasi-Cyclic, usually denoted as HQC, is a code-based Key Encapsulation Mechanism. Its security is based on the hardness of decoding random quasi-cyclic codes, more precisely on the Quasi-Cyclic Syndrome Decoding problem.

Differently from Classic McEliece, HQC does not rely on hiding a highly structured family of codes such as Goppa codes. Instead, it uses quasi-cyclic codes in order to reduce the size of the public key, while keeping the decoding problem hard.

The main idea is to work in the ring

$$R = \mathbb{F}_2[X]/(X^n - 1),$$

so that vectors of length n can be represented as polynomials modulo $X^n - 1$. In this representation, cyclic shifts correspond to multiplication by X. This allows HQC to exploit a quasi-cyclic structure and to describe large matrices in a compact way.

HQC uses two main building blocks:

- a random double-circulant code, used in the public-key encryption part;
- a decodable error-correcting code $\mathcal{C}$, instantiated through concatenated Reed–Muller and Reed–Solomon codes.

The Reed–Muller and Reed–Solomon concatenated code is used to correct the noise introduced during encryption. Thus, the correctness of HQC depends on the ability of this code to decode the resulting error vector.

5.6.2 Main algorithms

HQC is specified both as a public-key encryption scheme, denoted HQC-PKE, and as a Key Encapsulation Mechanism, denoted HQC-KEM.
The KEM version is obtained from the PKE version using the Fujisaki–Okamoto transformation.

1: Generate secret sparse vectors $x, y \in R$
2: Generate a public random vector $h \in R$
3: Compute
$$s = x + h \cdot y$$
4: Return the encryption key $ek_{PKE} = (h, s)$
5: Return the decryption key $dk_{PKE} = y$

HQC-PKE.Keygen

The public key contains the vectors needed to encrypt, while the secret key contains the information needed to remove the noise during decryption.

HQC-PKE.Encrypt

1: Encode the message m using the error-correcting code $\mathcal{C}$
2: Generate random sparse vectors $r_1, r_2, e \in R$
3: Compute
$$u = r_1 + h \cdot r_2$$
4: Compute
$$v = \mathcal{C}.\text{Encode}(m) + s \cdot r_2 + e$$
5: Return the ciphertext
$$c_{PKE} = (u, v)$$

HQC-PKE.Decrypt

1: Receive the ciphertext $c_{PKE} = (u, v)$
2: Compute
$$v - u \cdot y$$
3: Decode the result using $\mathcal{C}$.Decode
4: Return the recovered message m

The decryption works because

$$v - u \cdot y$$

removes the secret-dependent part of the ciphertext and leaves a noisy encoding of the message, which can be corrected by the code $\mathcal{C}$.

HQC-KEM

1: **KeyGen:** generate an HQC-PKE keypair
2: **Encaps:** choose a random message m, derive randomness, encrypt m, and derive the shared key K
3: **Decaps:** decrypt the ciphertext, recompute the encapsulation, and return the same shared key if the ciphertext is valid

The decapsulation algorithm includes a rejection step. If the recomputed ciphertext is different from the received ciphertext, the algorithm returns a pseudorandom rejection key. This is used to obtain IND-CCA2 security.

5.6.3 Particularities of HQC

- HQC is a code-based KEM based on quasi-cyclic syndrome decoding.
- The quasi-cyclic structure allows a reduction of the key size compared to generic code-based constructions.
- The scheme uses concatenated Reed–Muller and Reed–Solomon codes to correct decryption errors.
- HQC-PKE is IND-CPA secure, while HQC-KEM is IND-CCA2 secure thanks to the Fujisaki–Okamoto transformation.
- The main limitation is that HQC has larger ciphertexts and slower implementations compared to some lattice-based KEMs.

5.6.4 Comparison with other schemes

To make the comparison meaningful, we consider schemes providing approximately the same security level, namely about 256 bits of security or NIST security level 5.

First, we compare the public-key encryption schemes discussed above: RSA, McEliece, Niederreiter and HQC-PKE. RSA is compact, but it is not post-quantum secure. McEliece and Niederreiter are code-based and post-quantum secure, but they require large public keys. HQC-PKE reduces the public-key size by using a quasi-cyclic structure, at the cost of a larger ciphertext.

PKE Scheme	Security Level	Public Key	Secret Key	Ciphertext Size
RSA-15360	$\approx$ 256 bits	$\approx$ 1.9 KB	$\approx$ 4.75 KB	$\approx$ 1.9 KB
McEliece PKE	$\approx$ 256 bits	$\approx$ 6.4 MB	$\approx$ 14 KB	$\approx$ 1 KB
Niederreiter PKE	$\approx$ 256 bits	$\approx$ 1.6 MB	$\approx$ 14 KB	$\approx$ 0.2 KB
HQC-PKE-5	NIST-5	$\approx$ 7.1 KB	$\approx$ 0.03 KB	$\approx$ 14.1 KB

Table 2: Approximate comparison of PKE schemes at the 256-bit security level.

From the table, we can observe that HQC-PKE has a much smaller public key than McEliece and Niederreiter. However, this advantage is paid with a larger ciphertext. RSA remains very compact, but it is not secure against quantum attacks.

We can also compare the corresponding KEM constructions at the same security level. In this case, the comparison is between RSA-KEM, Classic McEliece KEM and HQC-KEM.

KEM Scheme	Security Level	Public Key	Secret Key	Ciphertext Size
RSA-KEM	$\approx$ 256 bits	$\approx$ 1.9 KB	$\approx$ 4.75 KB	$\approx$ 1.9 KB
Classic McEliece KEM	$\approx$ 256 bits	$\approx$ 1.2 MB	$\approx$ 14 KB	$\approx$ 0.2 KB
HQC-KEM-5	NIST-5	$\approx$ 7.1 KB	$\approx$ 7.2 KB	$\approx$ 14.1 KB

Table 3: Approximate comparison of KEM schemes at the 256-bit security level.

Thus, HQC-KEM can be seen as a compromise between the two extremes. It avoids the very large public key of Classic McEliece, while remaining post-quantum secure. The price to pay is a larger ciphertext and a higher computational cost during decapsulation.

6 Introduction to Lattice

29: Lattice

A lattice $\mathcal{L}$ is a discrete subgroup of $\mathbb{R}^n$.

Discrete means that, given an element of $\mathcal{L}$, there exists a neighborhood of such element that contains no other elements of $\mathcal{L}$. So, we can isolate our elemtns of the lattice.

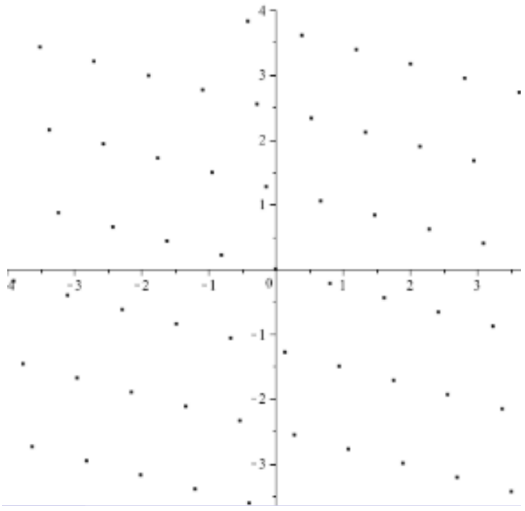


Figure 6: Lattice in $\mathbb{R}^2$

30: Rank

The rank of a lattice $\mathcal{L} \subset \mathbb{R}^n$, $rk(\mathcal{L})$ is the maximum number of linearly independent vectors in $\mathcal{L}$.
The lattice is called **full-rank** or **complete** if $\text{rk}(\mathcal{L}) = n$.
If $\mathcal{L} \subset \mathbb{Z}^n$ is called **integer lattice**.

Example

- $\mathbb{Z}^5$ is a lattice
- $\{(x, 2x) : x \in \mathbb{Z}\}$ is a lattice
- $\{(x, 2x) : x \in \mathbb{R}\}$ is not a lattice, because it is not discrete.
- $\{(2x, x + 1) : x \in \mathbb{Z}\}$ is not a lattice, because it is not a subgroup of $\mathbb{R}^2$.

6.1 Some properties of lattices

The intersection of two lattices is a lattice of $\mathbb{R}^n$ is a lattice in $\mathbb{R}^n$.

If $\mathcal{L}$ is a lattice in $c \in \mathbb{R}^x$ then

$$\mathcal{L}c = \{cv : v \in \mathcal{L}\} \text{ is a lattice in } \mathbb{R}^n.$$

This means that we procude a set by multiplying all elements of the lattice by a constant c

If $\mathcal{L}_1$ and $\mathcal{L}_2$ are lattices in $\mathbb{R}^n$ then

$$\mathcal{L}_1 + \mathcal{L}_2 = \{v_1 + v_2 : v_1 \in \mathcal{L}_1, v_2 \in \mathcal{L}_2\} \text{ is not in general a lattice in } \mathbb{R}^n.$$

That because we can lost the discreteness property.

31: Minimum Distance

Given a lattice $\mathcal{L}$, we define the minimum distance of $\mathcal{L}$, denoted by $\lambda(\mathcal{L})$, as the length of the shortest non-zero vector in $\mathcal{L}$

$$\lambda(\mathcal{L}) = \min\{\|v\| : v \in \mathcal{L} \setminus \{0\}\}$$

Let us observe that $\lambda(\mathcal{L}) = \min\{\|v\| : v \in \mathcal{L} \setminus \{0\}\} = \min\{\|v - w\| : v, w \in \mathcal{L}, v \neq w\}$

A lattice $\mathcal{L}$ of $\mathbb{R}^n$ is generated by a finite number of linearly independent vectors.

32: Basis

A basis $\mathcal{B}$ of a lattice $\mathcal{L}$ is a set of linearly independent generators of $\mathcal{L}$.
If $\mathcal{B} = \{b_1, b_2, \ldots, b_k\}$, then

$$\mathcal{L} = \{\textstyle\sum_{i=1}^{k} z_i b_i : z_i \in \mathbb{Z}\}$$

where k is the rank of $\mathcal{L}$.

Each lattice can has many different bases.

A basis can be rapresented by a $r \times n$ matrix

$$B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_r \end{bmatrix}$$

so that $\mathcal{L} = \{aB : a \in \mathbb{Z}^r\}$, where a is a row vector of $\mathbb{Z}^r$.

If B_1 and B_2 are two bases of the same lattice $\mathcal{L}$, then there exists a unimodular matrix U such that $B_1 = UB_2$.

NOT EVERY set of r linearly independent vectors in a lattice is a basis. This is true if the "parallelepiped" it determines does not contain any point of $\mathcal{L}$ except its vertices.

A lattice can have a proper sublattice of the same rank.

A lattice does not admit in general an orthogonal basis

6.1.1 Foundamental Domain

33: Foundamental Domain

Let $\mathcal{L}$ be a lattice of rank n with basis $\mathcal{B} = \{b_1, b_2, \ldots, b_n\}$.
The foundamental domain is

$$\mathcal{F}(\mathcal{B}) = \{\textstyle\sum_{i=1}^{n} \alpha_i b_i : 0 \leq \alpha_i < 1\}$$

34: Determinant

Given any basis B (in matricial form) of a lattice $\mathcal{L}$, the determinant of $\mathcal{L}$, denoted $\det(\mathcal{L})$, is the volume of the fundamental domain. If $\mathcal{L}$ is full rank, then $\det(\mathcal{L}) = |\det B|$.

Remark: The determinant is an invariant, i.e., it does not depend on the choice of the basis.

Example:
Consider the lattice generated by vectors $b_1 = (2, 1)$ and $b_2 = (1, -1)$, then the foundamental domain is the parallelogram determined by b_1 and b_2. [see Figure 7]
Lattice generated by the vectors $b_1 = (1, 0, 0)$, $b_2 = (0, 1, 0)$ and $b_3 = (0, 0, 1)$ the foundamental domain is the cube determined by b_1, b_2 and b_3. [see Figure 8]

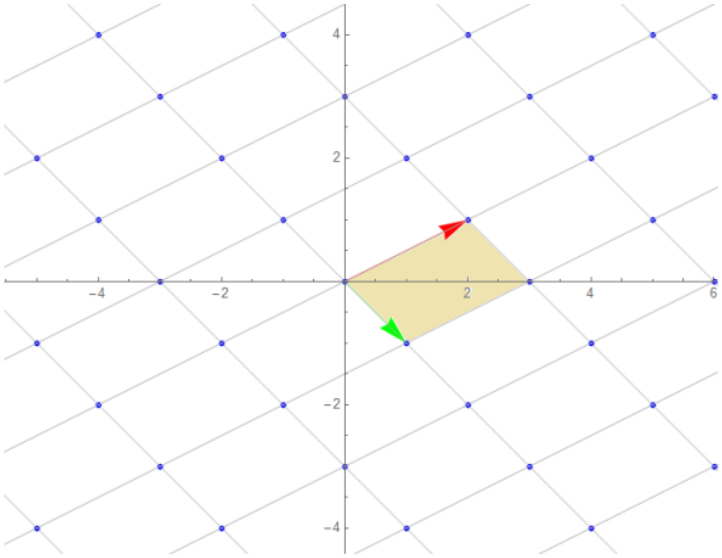


Figure 7: Foundamental Domain

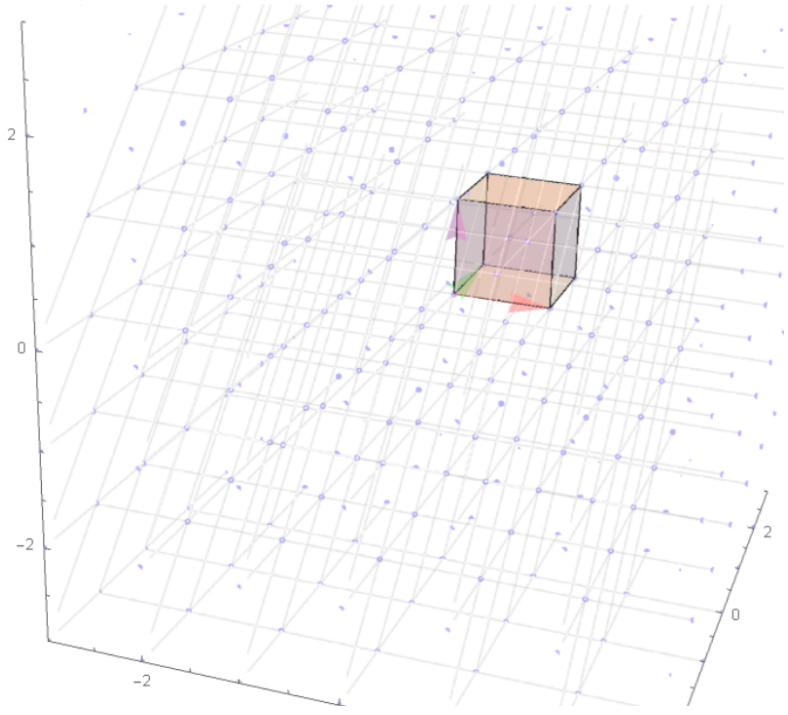


Figure 8: Foundamental Domain in $\mathbb{R}^3$

7: Minkowski's Bound

How long is a shortest nonzero vector in a lattice $\mathcal{L}$ of $\mathbb{R}^n$? The lattice $\mathcal{L}$ contains a nonzero vector v satisfying:

$$\|v\| \leq \sqrt{n} \cdot (\det(\mathcal{L}))^{1/n}$$

8

Let $\mathcal{L} \subset \mathbb{R}^n$ be a lattice of rank n, then $\forall v \in \mathbb{R}^n$ we have

$$v = b + f$$

for a unique $b \in \mathcal{L}$ and a unique $f \in \mathcal{F}$

It is possible to provide a one-to-one correspondence between lattice and polynomials by means of the following mapping:

$$\mathbb{Z}^n \leftrightarrow \mathbb{Z}[x]_{<n}, (a_0, a_1, \ldots, a_{n-1}) \leftrightarrow a_0 + a_1 x + \ldots + a_{n-1} x^{n-1}$$

An ideal lattice is a lattice $\mathcal{L} \subseteq \mathbb{Z}^n$ such that is isomorphic to an ideal $I \subseteq \mathbb{Z}_{[X]}/(f(x))$ with $f(x) \in \mathbb{Z}[x]$ monic and irreducible polynomial

6.2 Lattice Problems

We are interested to define some computational problems related to lattices, that are the basis of the security of lattice-based cryptography.

35: Shortest Vector Problem (SVP)

Given a lattice $\mathcal{L} \subseteq \mathbb{Z}^n$.
The Shortest Vector Problem (SVP) is the problem of finding a non-zero vector $v \in \mathcal{L}$ such that $\|v\| = \min_{x \in \mathcal{L}, x \neq 0} \|v\| = \lambda(\mathcal{L})$.

36: Approximated SVP problem (SVP-γ)

The approximated SVP problem (SVP-γ) is the problem of finding a vector $v \in \mathcal{L}$ such that $\forall x \in \mathcal{L}$:

$$\|v\| \leq \gamma \min \|x\|$$

where $\gamma \geq 1$ is an approximation factor.

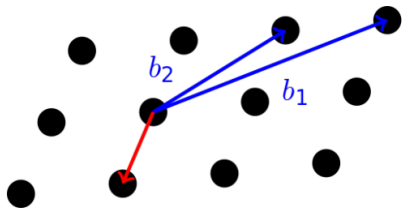


Figure 9: Shortest Vector Problem

37: Closest Vector Problem (CVP)

Given a lattice $\mathcal{L} \subseteq \mathbb{Z}^n$ and a target vector $w \in \mathbb{R}^n$.
The Closest Vector Problem (CVP) is the problem of finding a vector $v \in \mathcal{L}$ such that

$$\|v - w\| = \min_{x \in \mathcal{L}} \|x - w\|$$

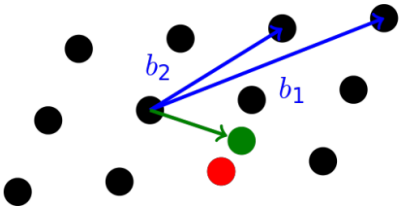


Figure 10: Closest Vector Problem

38: Approximated CVP problem (CVP-γ)

The approximated CVP problem (CVP-γ) is the problem of finding a vector $v \in \mathcal{L}$ such that $\forall x \in \mathcal{L}$:

$$\|v - w\| \leq \gamma \min \|x - w\|$$

where $\gamma \geq 1$ is an approximation factor.

SVP and CVP are computationally equivalent.

Orthogonal basis case

Both SVP and CVP are easy to solve if a lattice $\mathcal{L}$ of rank n has an orthogonal basis $B = \{b_1, \ldots, b_n\}$ (considering a full rank lattice). Indeed, if $a_i \in \mathbb{Z}$, $t_i \in \mathbb{R}$ for all $i = 1, \ldots, n$ and $v \in \mathcal{L}$ we have:

$$v = \sum_{i=1}^{n} a_i b_i \implies \|v\|^2 = \sum_{i=1}^{n} a_i^2 \|b_i\|^2$$
$$w = \sum_{i=1}^{n} t_i b_i \implies \|w - v\|^2 = \sum_{i=1}^{n} (a_i - t_i)^2 \|b_i\|^2$$

Therefore to solve SVP we search the shortest nonzero vector in the set $\{\pm b_1, \ldots, \pm b_n\}$ and to solve CVP we take each a_i as the integer closest to the corresponding t_i.

Lattices and Non-Orthogonal Bases

Consider a lattice $L \subseteq \mathbb{R}^n$ of rank 2 whose basis is $\underline{b}_1 = (5, 1, 1)$ and $\underline{b}_2 = (4, 0, 1)$. This is not an orthogonal basis, indeed:

$$\underline{b}_1 \cdot \underline{b}_2 = 5 \cdot 4 + 1 \cdot 0 + 1 \cdot 1 = 21 \neq 0$$

The lengths of these vectors are:

$$||\underline{b}_1|| = \sqrt{5^2 + 1^2 + 1^2} = \sqrt{27}$$

$$||\underline{b}_2|| = \sqrt{4^2 + 0^2 + 1^2} = \sqrt{17}$$

Let $\underline{v} = (v_1, v_2, \ldots, v_n) \in \mathbb{R}^n$. Remember that the length of a vector is:

$$||\underline{v}|| = \sqrt{\underline{v} \cdot \underline{v}} = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

The shortest vector of L is not, in this case, a vector of the basis. Indeed, for instance, $\underline{b}_1 - \underline{b}_2 \in L$ because it is a linear combination of $\underline{b}_1$ and $\underline{b}_2$ with integer coefficients. Since $\underline{b}_1 - \underline{b}_2 = (1, 1, 0)$, we have:

$$||\underline{b}_1 - \underline{b}_2|| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2}$$

which is smaller than the length of $\underline{b}_1$ and $\underline{b}_2$.

Thus, in general, given a lattice L and a non-orthogonal basis, the shortest vector is not necessarily a vector of the basis.

Lattices and Orthogonal Bases example

Consider a lattice $L \subseteq \mathbb{R}^n$ of rank 2 whose basis is given by $\underline{b}_1$ and $\underline{b}_2$.
Suppose that this is an orthogonal basis, i.e., $\underline{b}_1 \cdot \underline{b}_2 = 0$.
Now, we see that the shortest vector of L is a vector of the basis.

A general vector $\underline{v}$ is a linear combination of $\underline{b}_1$ and $\underline{b}_2$ with integer coefficients:

$$\underline{v} = a_1\underline{b}_1 + a_2\underline{b}_2 \quad \text{with } a_1, a_2 \in \mathbb{Z}$$

We consider the length of $\underline{v}$ squared (because it is more convenient):

$$\begin{aligned} ||\underline{v}||^2 &= \underline{v} \cdot \underline{v} = (a_1\underline{b}_1 + a_2\underline{b}_2) \cdot (a_1\underline{b}_1 + a_2\underline{b}_2) \\ &= a_1^2(\underline{b}_1 \cdot \underline{b}_1) + a_2^2(\underline{b}_2 \cdot \underline{b}_2) + a_1a_2(\underline{b}_1 \cdot \underline{b}_2) + a_2a_1(\underline{b}_2 \cdot \underline{b}_1) \end{aligned}$$

Since $a_1, a_2 \in \mathbb{Z}$ (and assuming $\underline{v} \neq \underline{0}$), we have $a_1^2 \geq 1$ or $a_2^2 \geq 1$, and consequently:

$$||\underline{v}||^2 \geq ||\underline{b}_1||^2 \quad \text{and} \quad ||\underline{v}||^2 \geq ||\underline{b}_2||^2$$

Thus the shortest vector of L can be $\underline{b}_1$ or $\underline{b}_2$.

This holds in general. Consider $L \subseteq \mathbb{R}^n$ of rank n with an orthogonal basis $\underline{b}_1, \ldots, \underline{b}_n$.
For all $\underline{v} \in L$, we have $\underline{v} = a_1\underline{b}_1 + \cdots + a_n\underline{b}_n$ with $a_i \in \mathbb{Z}$.

$$\begin{aligned} ||\underline{v}||^2 &= \underline{v} \cdot \underline{v} = (a_1\underline{b}_1 + \cdots + a_n\underline{b}_n) \cdot (a_1\underline{b}_1 + \cdots + a_n\underline{b}_n) \\ &= a_1^2(\underline{b}_1 \cdot \underline{b}_1) + \cdots + a_n^2(\underline{b}_n \cdot \underline{b}_n) = a_1^2||\underline{b}_1||^2 + \cdots + a_n^2||\underline{b}_n||^2 \end{aligned}$$

Thus, if L has an orthogonal basis, the shortest vector is a vector of the basis.

Closest Vector Problem

A similar situation happens if, given $\underline{w} \in \mathbb{R}^n$, we search for the closest vector of L to $\underline{w}$. In this case, we are searching for a vector $\underline{v} = a_1\underline{b}_1 + \cdots + a_n\underline{b}_n \in L$ (with $a_i \in \mathbb{Z}$) that minimizes $||\underline{w} - \underline{v}||$, or equivalently $||\underline{w} - \underline{v}||^2$.

Since $\underline{b}_1, \ldots, \underline{b}_n$ is a basis of L, it is also a basis of $\mathbb{R}^n$ (since they are linearly independent vectors), and:

$$\underline{w} = t_1\underline{b}_1 + \cdots + t_n\underline{b}_n \in \mathbb{R}^n \quad \text{with } t_i \in \mathbb{R}$$

We have $\underline{w} - \underline{v} = (t_1 - a_1)\underline{b}_1 + \cdots + (t_n - a_n)\underline{b}_n$ and:

$$\begin{aligned}
||\underline{w} - \underline{v}||^2 &= (\underline{w} - \underline{v}) \cdot (\underline{w} - \underline{v}) \\
&= \Big((t_1 - a_1)\underline{b}_1 + \cdots + (t_n - a_n)\underline{b}_n\Big) \cdot \Big((t_1 - a_1)\underline{b}_1 + \cdots + (t_n - a_n)\underline{b}_n\Big) \\
&= (t_1 - a_1)^2(\underline{b}_1 \cdot \underline{b}_1) + \cdots + (t_n - a_n)^2(\underline{b}_n \cdot \underline{b}_n) \\
&= (t_1 - a_1)^2||\underline{b}_1||^2 + \cdots + (t_n - a_n)^2||\underline{b}_n||^2
\end{aligned}$$

This quantity is minimized if and only if:

- a_1 is the closest integer to t_1
- a_2 is the closest integer to t_2
- ...
- a_n is the closest integer to t_n

6.2.1 Gram-Schmidt Orthogonalization

Given a basis $\underline{v}_1, \ldots, \underline{v}_n$ of $\mathbb{R}^n$, it transforms this into an orthogonal basis $\underline{u}_1, \ldots, \underline{u}_n$ of $\mathbb{R}^n$:

$$\begin{aligned}
\underline{u}_1 &= \underline{v}_1 \\
\underline{u}_2 &= \underline{v}_2 - \text{proj}_{\underline{u}_1}(\underline{v}_2)
\end{aligned}$$

Where $\text{proj}_{\underline{u}_1}(\underline{v}_2)$ is the orthogonal projection of $\underline{v}_2$ onto $\underline{u}_1$.

Let us check that $\underline{u}_2$ is orthogonal to $\underline{u}_1$. Geometrically, this is the sum (the diagonal of the parallelogram) between the vectors $\underline{v}_2$ and $-\text{proj}_{\underline{u}_1}(\underline{v}_2)$.

If a lattice $\mathcal{L}$ has a basis $\mathcal{B} = \{b_1, \ldots, b_n\}$ we searche for a good basis having shortest vector which are *"as orthogonal as possible to each others"*.

We can not directly use the correspondig Gram-Schmidt basis $\mathcal{B}^* = \{b_1^*, \ldots, b_n^*\}$: it is not a basis for $\mathcal{L}$, since the Gram-Schmidt process does not preserve the integrality of the coefficients.
However, $\mathcal{B}$ has a good property:

$$det(\mathcal{L}) = \prod_{i=1}^{n} ||b_i^*||$$

(since the change of the basis matrix from $\mathcal{B}$ to $\mathcal{B}^*$ has determinat equal to 1) and we could use $\mathcal{B}^*$ in oreder to find a new basis for $\mathcal{L}$ as similar as possible to $\mathcal{B}^*$

6.2.2 LLL algorithm

In a lattice $\mathcal{L}$ of rank n we have the **LLL algorithm** to reduce a basis $\mathcal{B} = \{v_1, \ldots, v_n\}$ with associated Gram-Schmidt orthogonal basis $\mathcal{B}^* = \{v_1^*, \ldots, v_n^*\}$ to a new basis with vectors as ortogonal as possible.
A basis is LLL-reduced if:

- **Size condition**:

$$|\mu_{i,j}| = \frac{|v_i, v_j^*|}{||v_j^*||^2} \leq \frac{1}{2} \quad \text{for all } 1 \leq j < i \leq n$$

- **Lovasz condition**:

$$||v_i^*||^2 \geq \left(\frac{3}{4} - \mu_{i,j}^2\right) ||v_{i-1}^*||^2 \quad \text{for all } 2 \leq i \leq n$$

Algorithm 6 Lenstra–Lenstra–Lovász (LLL) Reduction Algorithm

Require: Basis $\mathcal{B} = \{v_1, \ldots, v_n\}$ of an n-dimensional lattice L
Ensure: LLL-reduced basis $\mathcal{B}_{LLL} = \{v_1, \ldots, v_n\}$

```
k ← 2
v_1^* ← v_1
while k ≤ n do
    for j = k − 1, k − 2, ..., 1 do
        v_k ← v_k − ⌊μ_{k,j}⌉ v_j                    ▷ Size Reduction
    end for
    if ‖v_k^*‖² ≥ (3/4 − μ_{k,k−1}²) ‖v_{k−1}^*‖² then
        k ← k + 1
    else
        Swap v_{k−1} and v_k
        k ← max{k − 1, 2}                            ▷ Swap
    end if
end while
return B_LLL
```

Note:

$$\mu_{k,j} = \frac{\langle v_k, v_j^* \rangle}{\|v_j^*\|^2}$$

where the vectors $v_1^*, \ldots, v_k^*$ are computed using the Gram–Schmidt orthogonalization process.

6.3 Learning With Errors – LWE

39: Learning With Errors (LWE)

Let p be a prime and let $n \in \mathbb{N}^+$.
Given a secret vector $s \in \mathbb{Z}_p^n$.
The **LWE-distribution** $A_{s,\chi}$ is defined as:

$$A_{s,\chi} = \{(a, b) \in \mathbb{Z}_p^n \times \mathbb{Z}_p$$

such that:

- a is chosen uniformly at random from $\mathbb{Z}_p^n$ (vector of lenght a)
- b is defined as:

  $$b = s \cdot a + e \mod p = (\langle s, a \rangle + e) \mod p$$

  where e is an error term chosen according to a distribution χ over $\mathbb{Z}_p$.
  So b is a a scalar, dot product between s and a plus an error term

LWE are typcally hard problems. In particular:

- **Search LWE:** The problem Search $LWE_{n,p,\chi,m}$ is to find $s \in \mathbb{Z}_p^n$ given m pairs $(a, b) \in A_{s,\chi}$
- **Decision LWE:** The problem Decision $LWE_{n,p,\chi,m}$ is to say, given m pairs $(a, b) \in A_{s,\chi}$ and m pairs (a, b) where a if they belong to a LWE distribution with a same fixed s or if they are randomly sampled from a uniform distribution in $\mathbb{Z}_p^n \times \mathbb{Z}_p$.

The two problems are computationally equivalent.

In cryptography, is more convinient to work with the Ring-LWE problem.

40: Ring Learning With Errors (Ring-LWE)

Consider a prime p and $n \in \mathbb{N}^+$, define $R = \mathbb{Z}_p[X]/(X^n + 1)$.
Let $s = s(X) \in R$ a fixed element called secret, wich is a polynomial of degree at most $n - 1$ with coefficients in $\mathbb{Z}_p$.
The **Ring-LWE distribution** $A_{s,\chi}$ is defined as:

$$A_{s,\chi} = \{(a, b)\} \subseteq R \times R$$

such that:

- $a = a(X)$ is chosen uniformly at random from R
- $b = b(X)$ is defined as:

$$b = (s \cdot a + e) \mod p$$

  where $e = e(X)$ is an error term chosen according to a distribution χ over R.

Also Ring-LWE is a hard problem:

- **Search Ring-LWE:** The problem Search $RingLWE_{n,p,\chi,m}$ is to find $s \in R$ given m pairs $(a, b) \in A_{s,\chi}$
- **Decision Ring-LWE:** The problem Decision $RingLWE_{n,p,\chi,m}$ is to say, given m pairs $(a, b) \in A_{s,\chi}$ if they come from a Ring-LWE distribution with a same fixed s or if they are randomly sampled from a uniform distribution in $R \times R$.

7 Lattice-based cryptography

7.1 NTRU

> ***Author's note:*** *As I already said, I strongly suggest to read the original paper, because the professor only gives a very high level overview of the scheme, and it's not possible to understand the details of the scheme without reading the original paper. Moreover, this paragraph will contain the fondumental parts needed aswer to a possible question in the exam.*

7.1.1 Parameters and notation

N,p,q integers with gcd(p,q)=1 and q>p.
(p and q are not necessarily prime and N$\neq$ pq)

$R = \mathbb{Z}[X]/(X^N - 1)$ is the set of polynomials woth integer coefficients and degree at most N-1, they are all the possible reminders with respect to the division by $X^N - 1$.
The product in R will be denoted by $\circledast$

$\mathcal{L}_f, \mathcal{L}_g, \mathcal{L}_\phi, \mathcal{L}_m$ are subsets of R.

Remark

In R, $X^N = 1$ or equivalently $X^N \equiv 1 \mod (X^N - 1)$
Indeed, $X^N = 1 \cdot (X^N - 1) + 1$ and the reminder of the division of X^N by $X^N - 1$ is 1.

Example

Consider N=4 and F,G $\in$ R:

$$F = F_0 + F_1X + F_2X^2 + F_3X^3$$
and
$$G = G_0 + G_1X + G_2X^2 + G_3X^3$$

We have:

$$
\begin{aligned}
&F \circledast G = (F_0 + F_1X + F_2X^2 + F_3X^3) \\
&\circledast (G_0 + G_1X + G_2X^2 + G_3X^3) = \\
&= F_0G_0 + F_0G_1X + F_0G_2X^2 + F_0G_3X^3 + F_1 \\
&G_0X + F_1G_1X^2 + F_1G_2X^3 + F_1G_3X^4 + \\
&F_2G_0X^2 + F_2G_1X^3 + F_2G_2X^4 + F_2G_3X^5 + \\
&F_3G_0X^3 + F_3G_1X^4 + F_3G_2X^5 + F_3G_3X^6 =
\end{aligned}
$$

But we have that $X^4 = 1$ so:

$$
\begin{aligned}
&= F_0G_0 + F_1G_3 + F_2G_2 + F_3G_1 + \\
&(F_0G_1 + F_1G_0 + F_2G_3 + F_3G_2)X + \\
&(F_0G_2 + F_1G_1 + F_2G_0 + F_3G_3)X^2 + \\
&(F_0G_3 + F_1G_2 + F_2G_1 + F_3G_0)X^3
\end{aligned}
$$

7.1.2 Key Generation

1: Select $f \in \mathcal{L}_f \subseteq R$ s.t f is invertible in R mod p and mod q
2: Select $g \in \mathcal{L}_g \subseteq R$
3: Evaluate F_p and F_q the inverse of f in R mod p and mod q
4: Evaluate $h \equiv F_q \circledast g \mod q$

7.1.3 Encryption

Encryption of a message $m \in \mathcal{L}_m \subseteq R$.

1: Select random $\phi \in \mathcal{L}_\phi$
2: Evaluate $c \equiv p \cdot \phi \circledast h + m \mod q$

The one-way function is based on R-LWE

Decryption

1: Evaluate $a \equiv f \circledast c \mod q$
2: Evaluate $F_q \circledast a \mod p \equiv m$

Note: In the reduction mod q (and mod p), we use the rapresentation in the inteval $[-\frac{q}{2}, \frac{q}{2}]$ (and $[-\frac{p}{2}, \frac{p}{2}]$) to avoid the wrap around effect.
For example, if we have $q = 5$, we do not use 0,1,2,3,4 but we use $\{-2, -1, 0, 1, 2\}$.

The centered representation modulo q does not prevent the modular reduction itself.
Rather, it allows us to interpret residues modulo q as small signed coefficients.
This is crucial in NTRU, because during decryption the intermediate coefficients are expected to be small.
If they remain inside the interval $[-\frac{q}{2}, \frac{q}{2}]$, then the centered reduction recovers their correct signed value.
If a coefficient exceeds this interval, a wrap around occurs and the value may be interpreted incorrectly, possibly causing decryption failure.

E.g. Let us consider $q = 11$. The standard representation modulo 11 is

$$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\},$$

whereas the centered representation is

$$\{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5\}.$$

Therefore, the residues

$$6, 7, 8, 9, 10$$

are interpreted in the centered representation as

$$6 \equiv -5 \pmod{11}, \qquad 7 \equiv -4 \pmod{11},$$

$$8 \equiv -3 \pmod{11}, \qquad 9 \equiv -2 \pmod{11}, \qquad 10 \equiv -1 \pmod{11}.$$

For example, suppose that after an NTRU computation we obtain the following small coefficients:

$$a = (-1, -4, 0, 2, -5).$$

All these coefficients belong to the centered interval

$$[-5, 5].$$

However, if the coefficients are represented modulo 11 in the standard way, we obtain

$$a \bmod 11 = (10, 7, 0, 2, 6),$$

because

$$-1 \equiv 10 \pmod{11}, \qquad -4 \equiv 7 \pmod{11}, \qquad -5 \equiv 6 \pmod{11}.$$

If we read the vector as

$$(10, 7, 0, 2, 6),$$

we may incorrectly interpret some coefficients as large positive values. Instead, using the centered representation, we recover

$$(10, 7, 0, 2, 6) \longrightarrow (-1, -4, 0, 2, -5).$$

Thus, the centered representation allows us to correctly interpret residues modulo q as small signed coefficients, which is crucial in NTRU-based schemes.

Proof of correctness

$$a \equiv f \circledast c \mod q \equiv f \circledast (p \cdot \phi \circledast h + m) \mod q \equiv p \cdot \phi \circledast g + f \circledast m \mod q$$

In the last equality we exploit the secret relation between the public and private keys. In particular the fact that $h \equiv F_q \circledast g \mod q$ is the trapdoor.

For appropriate choices of the parameters, we can ensure that the polynomial $a = p \cdot \phi \circledast g + f \circledast m$ is already reduced mod q, i.e., its coefficients are in the interval $[-\frac{q}{2}, \frac{q}{2}]$.
So that, the polynomial a does not change if it is reduced mod q (we don't have the wrap around effect).

Then, when we evaluate $F_q \circledast a \mod p$, we have

$$F_q \circledast (p \cdot \phi \circledast g + f \circledast m) \mod p \equiv F_q \circledast f \circledast m \mod p \equiv m$$

In order to recover original message m, we need that the coefficients of m were already reduced mod p.

Remark

Consider the ciphertext $c \equiv p \cdot \phi \circledast h + m \mod q$.
If we reduce $c \mod p$ we do not obtain m, because the polynomila $p \cdot \phi \circledast h + m$ has not the coeffincients in the interval $[-\frac{p}{2}, \frac{p}{2}]$ (we have the wrap around effect), but they are larger, they wrap aroud mod q and consequently we do not find the original message m.
In general we gave the following situation:

- $p \cdot \phi \circledast g + f \circledast m \mod q = p \cdot \phi \circledast g + f \circledast m \in \mathbb{Z}_{[X]}/(X^N - 1)$
- $p \cdot \phi \circledast h + m \mod q \neq p \cdot \phi \circledast h + m \in \mathbb{Z}_{[X]}/(X^N - 1)$

It is clearly visible in the following example:

Example:

Consider $N = 4$, $p = 7$ and $q = 19$.
Suppose

$$m = x^3 + x^2 + x + 1, \quad f = x^3 + x^2 + 1, \quad \phi \circledast g = 2x^3 + 2x^2 + x + 2$$

Then we have:

$$F_p = 4x^3 + 5x^2 + 5x + 5, \quad F_q = 12x^3 + 13x^2 + 13x + 13 \quad \text{and} \quad f \circledast m = 3x^3 + 3x^2 + 2x + 3$$

Now we compute:

$$\begin{aligned} p \cdot \phi \circledast g + f \circledast m &= 14x^3 + 14x^2 + 7x + 14 + 3x^3 + 3x^2 + 3x + 3 \\ &= 17x^3 + 17x^2 + 10x + 17 \end{aligned}$$

Which is the same polynomial after the reduction mod q:

$$p \cdot \phi \circledast g + f \circledast m \mod q = 17x^3 + 17x^2 + 10x + 17$$

Thus, if we reduce the coefficients mod p, we get $f \circledast m = 17x^3 + 17x^2 + 10x + 17 \mod p = 3x^3 + 3x^2 + 3x + 3$

Let us see what happens to $c = p \cdot \phi \circledast g \circledast F_q + m$:

$$\phi \circledast g \circledast F_q = 89x^3 + 89x^2 + 89x + 90$$

and

$$\begin{aligned} p \cdot \phi \circledast g \circledast F_q + m &= 623x^3 + 623x^2 + 623x + 630 + 3x^3 + 3x^2 + 3x + 3 = \\ &626x^3 + 626x^2 + 626x + 633 \end{aligned}$$

And this polynomial changes after the reduction mod q:

$$p \cdot \phi \circledast g \circledast F_q + m \mod q = 18x^3 + 18x^2 + 18x + 6$$

Thus, if we reduce the coefficients mod p, we DO NOT get m:

$$18x^3 + 18x^2 + 18x + 6 \mod p = 4x^3 + 4x^2 + 4x + 6$$

This is due to the fact that

$$623 \equiv 0 \mod 7, \quad 630 \equiv 0 \mod 7$$

but

$$623 \equiv 15 \mod 19 \quad \text{and} \quad 15 \not\equiv 0 \mod 7$$
$$630 \equiv 3 \mod 19 \quad \text{and} \quad 3 \not\equiv 0 \mod 7$$

7.1.4 Attacks against NTRU

Brute force

We want that $|\mathcal{L}_f|, |\mathcal{L}_g|, |\mathcal{L}_\phi|$ are large enough to make the brute force attack infeasible.

Considering that $|\mathcal{L}_g| < |\mathcal{L}_f|$, let us evaluate the number of polynomials in $\mathcal{L}_g = \mathcal{L}(d_g, d_g)$ (remember that the polynomials in $\mathcal{L}_g$ are ternary polynomials with d_g coefficients equal to 1, d_g coefficients equal to -1 and the rest of the coefficients equal to 0)

Thus:

$$|\mathcal{L}_g| = \binom{N}{d_g} \cdot \binom{N - d_g}{d_g} \cdot 1 = \frac{N!}{d_g!\cancel{(N - d_g)!}} \cdot \frac{\cancel{(N - d_g)!}}{d_g!(N - 2d_g)!} = \frac{N!}{d_g!^2(N - 2d_g)!}$$

Multiple Transmission Attack

If we use the same publick key h to encrypt several times the same message m, then an attacker can recover a sufficient number of coefficients of the random polynomial used during the encryption and then use **brute froce** to recover the original message.

Indeed, if we have

$$\begin{aligned} c_1 &\equiv p \cdot \phi_1 h + m \mod q \\ c_2 &\equiv p \cdot \phi_2 h + m \mod q \\ &\vdots \\ c_k &\equiv p \cdot \phi_k h + m \mod q \end{aligned}$$

then the attacker can compute:

$$c_1 - c_2 \equiv p \cdot h(\phi_2 - \phi_1) \mod q$$

from which

$$\phi_2 - \phi_1 \equiv p^{-1} h^{-1} (c_1 - c_2) \mod q$$

Thus, and attacker discovers $\phi_2 - \phi_1$ and similarly $\phi_3 - \phi_1, \phi_4 - \phi_1, \ldots, \phi_k - \phi_1$.

Thanks to this polynomial can recover some coefficients of ϕ_1 when he observe the coefficients $\neq 0, 1, -1$ in the polynomials $\phi_2 - \phi_1, \phi_3 - \phi_1, \ldots, \phi_k - \phi_1$.

Example

Remember that the polynomials $\phi_1, \phi_2, \ldots, \phi_k$ are ternary polynomials. Suppose that $N = 8$ and

$$\begin{aligned} \phi_1 &= x^5 + x^3 + 1 \\ \phi_2 &= x^7 + x^6 - x^4 + x^3 + x \\ \phi_3 &= -x^5 + x^4 + x^2 - x \end{aligned}$$

Then we have:

$$\begin{aligned} \phi_2 - \phi_1 &= x^7 + x^6 - x^5 - x^4 + 2x^3 + x - 1 \\ \phi_3 - \phi_1 &= -2x^5 + x^4 + x^3 + x^2 - 2x + 1 \end{aligned}$$

If we only know these, we are sure that the coeffient of x^3 in ϕ_1 must be -1 (otherwise it is impossible that $\phi_2 - \phi_1$ has a coefficient of 2 for x^3)
Similarly, we are sure that the coefficient of x^5 and x in ϕ_1 must be 1

Lattice based attack

Consider:

$$f = f_0 + f_1 x + \ldots + f_{N-1} x^{N-1}$$
$$g = g_0 + g_1 x + \ldots + g_{N-1} x^{N-1}$$
$$h = h_0 + h_1 x + \ldots + h_{N-1} x^{N-1}$$

and the corresponding vectors:

$$\underline{f} = (f_0, f_1, \ldots, f_{N-1})$$
$$\underline{g} = (g_0, g_1, \ldots, g_{N-1})$$
$$\underline{h} = (h_0, h_1, \ldots, h_{N-1})$$

The vector $(\alpha \underline{f}, \underline{g}) = (\alpha f_0, \alpha f_1, \ldots, \alpha f_{N-1}, g_0, g_1, \ldots, g_{N-1})$ is a short vector in the lattice of rank 2N whose basis is obtained exploiting the public key, for a convenient choice of α.

Example:

Consider $N = 3$ and the lattice whose basis is given by the rows of the following 6x6 matrix:

$$\begin{pmatrix} \alpha & 0 & 0 & h_0 & h_1 & h_2 \\ 0 & \alpha & 0 & h_2 & h_0 & h_1 \\ 0 & 0 & \alpha & h_1 & h_2 & h_0 \\ 0 & 0 & 0 & q & 0 & 0 \\ 0 & 0 & 0 & 0 & q & 0 \\ 0 & 0 & 0 & 0 & 0 & q \end{pmatrix}$$

we have that

$$f_0(\alpha, 0, 0, h_0, h_1, h_2) + f_1(0, \alpha, 0, h_2, h_0, h_1) + f_2(0, 0, \alpha, h_1, h_2, h_0)$$
$$= (\alpha f_0, \alpha f_1, \alpha f_2, f_0 h_0 + f_1 h_2 + f_2 h_1, f_0 h_1 + f_1 h_0 + f_2 h_2, f_0 h_2 + f_1 h_1 + f_2 h_0)$$

Note that the last three coordinates of this vector are equal to g_0, g_1, g_2 respectively.

Indeed, remember that

$$h = F_q \circledast g \mod q \iff F_q \circledast h \equiv g \mod q$$

7.1.5 Size comparison

	PL	CT	RATIO	PK	SK	SECURITY
RSA	2048	2048	1	2048	2048	100
MCELIECE	524	1024	1.9	$524 \cdot 1024$	$524^2 + 524 \cdot 1024 + 8749$	65
NIED	284	500	1.76	$500 \cdot 1024$	$500^2 + 500 \cdot 1024 + 8749$	65

Table 4: Size of plaintexts, ciphertexts and keys in bits.

NTRU, Case B, $N = 167$, $p = 3$, $q = 128$, $R = \mathbb{Z}[X]/(X^N - 1)$

- **Plaintext:** $m \in R$ with coefficients reduced modulo p:
  $\lceil \log_2 p \rceil \cdot N = 334$ bits
- **Ciphertext:** $c \in R$ with coefficients reduced modulo q:
  $\lceil \log_2 q \rceil \cdot N = 1169$ bits
- **Ratio:** $\frac{1169}{334} = 3.5$
- **Public key:** $h \in R$ with coefficients reduced modulo q:
  $\lceil \log_2 q \rceil \cdot N = 1169$ bits
- **Private key:** $f, F_p \in R$ with coefficients reduced modulo p:
  $2 \cdot \lceil \log_2 p \rceil \cdot N = 668$ bits

NOTE: Remember that for representing an integer N, we need $\lceil \log_2 N \rceil$ bits.

7.2 Falcon

Author's note: *Also in this case, I strongly suggest to read the original paper.*

Before describing Falcon scheme, we need to introduce some mathematical tools.
In particular it is necessary to introduce the concept of **Cyclotomic Polynomials**.

7.2.1 Cyclotomic Polynomials

The roots of the polynomial $X^N - 1$ are called N-th roots of unity and we denote them by $z_0, z_1, \ldots, z_{N-1}$.

We can explicitly write the complex number:

$$z_k = e^{\frac{2k\pi i}{N}} = \cos\left(\frac{2k\pi}{N}\right) + i\sin\left(\frac{2k\pi}{N}\right)$$

for $k = 0, 1, \ldots, N-1$.

Indeed $(z_k)^N = e^{2k\pi i} = \cos 2k\pi + i\sin 2k\pi = 1$

The set of ALL N-th roots of unity

$$M_N = \{z_0, z_1, \ldots, z_{N-1}\}$$

is a cycling group of order N and the generators of M_N are called N-th primitive roots of unity.

Recalling that a cycling group of order N is isomorphic to the additive group $(\mathbb{Z}_N, +)$, we have that:

- Number of generators of Z_N is $\varphi(N)$,
  so the number of generators of M_N is $\varphi(N)$, i.e., the number of N-th primitive roots of unity is $\varphi(N)$
- The generator of Z_N are all and only the integers k comprime with N so z_k is a primitive root of unity if and only if k is coprime with N (i.e., $\gcd(k, N) = 1$)

41: N-th Cyclotomic Polynomial

The N-th cyclotomic polynomial $\Psi_N(x)$ is the monic polynomial whose roots are all and only the N-th primitive roots of unity, i.e., $\Psi_N(x) = \prod_{k:\gcd(k,N)=1}(x - z_k)$

Remark: $\deg \Psi_N(x) = \varphi(N)$

9

$$x^N - 1 = \prod_{d|N} \Psi_d(x)$$

Idea of Proof:

- for all $d|N$, the roots of $\Psi_d(x)$ are also roots of $x^N - 1$ (because they are N-th roots of unity)
  Indeed, let α be a root of $\Psi_d(x)$, i.e., $\alpha^d = 1$
  then, $\alpha^N = \alpha^{md} = (\alpha^d)^m = 1^m = 1 (d \mid N \implies N = md)$
- It is also necessary to prove that

$$deg \prod_{d|N} \Psi_d(x) = N \iff \sum_{d|N} \varphi(d) = N$$

Now, let us consider

$N = 1, \quad , x - 1 = \Psi_1(x)$

$N = 2, \quad x^2 - 1 = \Psi_1(x) \cdot \Psi_2(x) = (x-1)\Psi_2(x) \implies \boxed{\Psi_2(x) = x + 1}$

$N = 3, \quad x^3 - 1 = \Psi_1(x) \cdot \Psi_3(x) = (x-1)\Psi_3(x) \implies \boxed{\Psi_3(x) = x^2 + x + 1}$

$N = 4, \quad x^4 - 1 = \Psi_1(x) \cdot \Psi_2(x) \cdot \Psi_4(x) = (x-1)(x+1)\Psi_4(x) \implies \boxed{\Psi_4(x) = x^2 + 1}$

$\vdots$

$N = 8, \quad x^8 - 1 = \Psi_1(x) \cdot \Psi_2(x) \cdot \Psi_4(x) \cdot \Psi_8(x) = (x-1)(x+1)(x^2+1)\Psi_8(x)$

$\implies \boxed{\Psi_8(x) = x^4 + 1}$

$\vdots$

$N = 2^k, \quad x^{2^k} - 1 = \Psi_1(x) \cdot \Psi_2(x) \ldots \Psi_{2^k}(x) = (x-1)(x+1)(x^2+1)(x^4+1)\cdots(x^{2^{k-1}}+1)\Psi_{2^k}($

$\implies \boxed{\Psi_{2^k}(x) = x^{2^{k-1}} + 1}$

7.2.2 Genealogy of Falcon

Falcon is the product of many years of work, not only by the authors but also by others. This section explains how these works gradually led to Falcon as we know it.

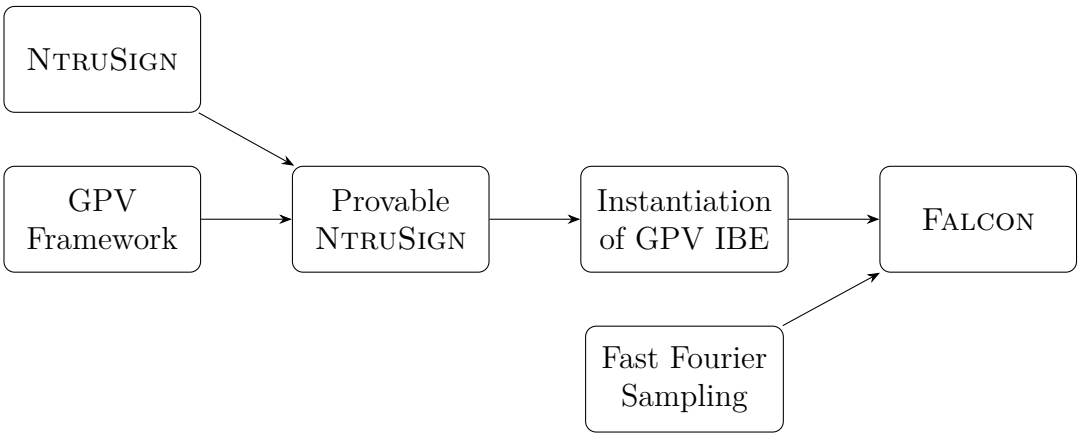


Figure 11: Genealogy of FALCON

7.2.3 The Gentry-Peikert-Vaikuntanathan (GPV) Framework

The GPV framework establishes a framework for constructing a secure lattice-based signature scheme.

- The public key contain a full-rank matrix $A \in \mathbb{Z}_q^{n \times m}$ (where m > n) generating a q-ary lattice Λ
- The private key contains a matrix $B \in \mathbb{Z}_q^{m \times m}$ generating $\Lambda_q^\perp$
  where $\Lambda_q^\perp$ denotes the lattice ORTOGONAL to Λ: $\forall x \in \Lambda$ and $y \in \lambda_q^\perp$, we have $\langle x, y \rangle \equiv 0 \mod q$ (where $\langle x, y \rangle$ is the standard inner product between the vectors x and y)
  Equivalently, the rows of A and B are pairwise orthogonal: $BxA^t = 0$
- Given a message m, a signature of m is a short vector $s \in \mathbb{Z}^m$ such that $sA^t = H(m)$, where H is a hash function $(\{0,1\}* \to \mathbb{Z}_q^n)$
- Given A, verification of a signature s for a message m consists in checking that $sA^t = H(m)$ and that s is short enough (i.e., $\|s\| \leq \beta$ for some parameter β)
  So it is straightforward

- Computing a valid signature is more difficult.
  First, compute $c_0 \in \mathbb{Z}_q^m$ which verifies $c_0 A^t = H(m)$
  Even if c_0 is not short, we can simplify it with linear algebra.
  B is then used to compute a vector $v \in \Lambda_q^\perp$ close to c_0
  The difference $s = c_0 - v$ is a valid signature: indeed, $sA^t = c_0 A^t - vA^t = c - 0 = H(m)$ and s is short because v is close to c_0.

Note that B is consider a good basis of $\Lambda_q^\perp$ if the vectors in B are short and close to orthogonal.

7.2.4 NTRU lattice

As already said, Falcon exploits the NTRU lattice.

Let $\phi = x^n + 1$ for $n = 2^k$.
A set of NTRU secret consist in 4 polynomials $f, g, F, G \in \mathbb{Z}[X]/(\phi)$ such that

$$fG - gF = q \mod \phi \text{ (NTRU equation).}$$

Provided that f is invertible modulo q, we can define the public key as $h \leftarrow g \cdot f^{-1} \mod q$ and the private key as the tuple (f, g, F, G).
Indeed, one can check that the matrices

$$\left[\begin{array}{c|c} 1 & h \\ \hline 0 & q \end{array}\right] \quad \text{and} \quad \left[\begin{array}{c|c} f & g \\ \hline F & G \end{array}\right]$$

generate the same lattice, but the first one contains two large polynomials (h and q) while the second one contains four small polynomials.

If f, g are generated with enough entropy, then h will look pseudorandom. However in practice, even when f, g are quite small, it remains hard to find small polynomials f', g' such that $h = g' \cdot (f')^{-1} \mod q$. The hardness of this problem constitutes the **NTRU assumption**.

Instantiation with the GPV framework

We now instantiate GPV.

- The public basis is the matrix $A = [1|h^*]$ where h^* is the vector of the coefficients of h (i.e., if $h = h_0 + h_1 x + \ldots + h_{n-1} x^{n-1}$, then $h^* = (h_0, h_1, \ldots, h_{n-1})$)

- The secret basis is the matrix $B = \left[\begin{array}{c|c} g & -f \\ \hline G & -F \end{array}\right]$ One can check that A and B are indeed orthogonal:$BxA^* = 0 \mod q$
- The signature of a message of a message m is composed by two polynomials s_1, s_2 such that $s_1 + s_2 h = H(m|r)$, where r is a random salt. Note that, since s_1 is completely determined by m, r, s_2, it is sufficient to store only (s_2, r) in the signature.

Computation:
$A* = [1|h]$

$$B \cdot A^* = \left[\begin{array}{c|c} g & -f \\ \hline G & -F \end{array}\right] \cdot \left[\frac{1}{h}\right] = \left[\frac{g - fh}{G - Fh}\right] = \left[\frac{g - g}{G - G}\right] = 0$$

Since the public key is defined as

$$h \equiv gf^{-1} \pmod{q},$$

we have

$$fh \equiv fgf^{-1} \equiv g \pmod{q}.$$

Moreover, from the NTRU equation

$$fG - gF \equiv 0 \pmod{q},$$

we obtain

$$fG \equiv gF \pmod{q}.$$

Multiplying both sides by f^{-1}, we get

$$G \equiv Fgf^{-1} \equiv Fh \pmod{q}.$$

Therefore, we get 0 as expected

7.2.5 Fast Fourier Sampling

The authors exploit the Fast Fourier Transform (FFT) to speed up the sampling of the signature.

The idea is to represent the polynomials in the evaluation form, i.e, as vectors of the evaluations of the polynomials at the N-th roots of unity.
So instead of representing a polynomial $a = a_0 + a_1 x + \ldots + a_{n-1} x^{n-1}$ as the vector of its coefficients $(a_0, a_1, \ldots, a_{n-1})$, we represent it as the vector of its evaluations at the N-th roots of unity $(a(z_0), a(z_1), \ldots, a(z_{n-1}))$. where $z_0, z_1, \ldots, z_{n-1}$ are the N-th roots of unity (complex numbers).

7.2.6 Falcon algorithms

> ***Author's note:*** *This section is made to simplify the description of the main algorithms. For a more detailed description, I suggest to read the original paper.*

Remember that Falcon fix:

$$q, n \in \mathbb{Z}, \text{ where } n = 2^k$$
$$\phi = x^n + 1$$
$$R = \mathbb{Z}[X]/(\phi)$$

Key Generation

1: Sample $f, g \in R$ s.t $(||f||, ||g||) < 1.17\sqrt{q}$
2: Compute $F, G \in R$ s.t. $fG - gF = q$ (NTRU equation)
3: Define B as the matrix $\left[\begin{array}{c|c} g & -f \\ \hline G & -F \end{array}\right]$ good basis of lattice $\Lambda_q^{\perp}$. Then we compute $\hat{B}$ applying the FFT to each element of B
4: Compute $h = g \cdot f^{-1} \mod q$ and define $A = [1|h^*]$ where h^* is the vector of the coefficients of h
5: Return $pk = h$ and $sk = (f, g, F, G)$

In order to compute F, G we can reduce the problem over the integers, exploiting two operators:

- **Split:**

  $$A(x) = a_0 + a_1 x + \ldots + a_{n-1} x^{n-1} \xrightarrow{\text{Split}} (A_0(x), A_1(x))$$

  where $A_0(x)$ contains the powers of x with even exponent and $A_1(x)$ contains the powers of x with odd exponent.
  For example, if $A(x) = 3 + 2x + 5x^2 + 7x^3$, then $A_0(x) = 3 + 5x$ and $A_1(x) = 2 + 7x$
  Iterating the split operator, we can split a polynomial in R into n polynomials of degree 0, i.e., into n integers.

- **Merge:**

  $$(A_0(x), A_1(x)) \xrightarrow{\text{Merge}} A(x) = A_0(x^2) + xA_1(x^2)$$

  For example, if $A_0(x) = 3 + 5x$ and $A_1(x) = 2 + 7x$, then $A(x) = A_0(x^2) + xA_1(x^2) = 3 + 5x^2 + 2x + 7x^3 = 3 + 2x + 5x^2 + 7x^3$

Signature

1: Compute $H(r \mid m) = c$ where $c \in R$
2: Observe that

$$(\underline{c}, \underline{0})A^* = c$$

where

$$A^* = [1 \mid h]$$

3: Compute

$$t = (\underline{c}, \underline{0})B^{-1}$$

such that

$$\underline{t}B = (\underline{c}, \underline{0})$$

4: Find a lattice vector z generated by B close to t
5: Define

$$\underline{s} = (s_1, s_2) = (\underline{t} - \underline{z})\hat{B}$$

where $\hat{B}$ is the FFT representation of B
6: Observe that $\underline{s}$ is short and satisfies:

$$\underline{s}A^* = (\underline{t} - \underline{z})\hat{B}A^*$$

7: Since the rows of B are orthogonal to A^*, we have:

$$z\hat{B}A^* = 0$$

8: Therefore:

$$\underline{s}A^* = t\hat{B}A^* = (\underline{c}, \underline{0})B^{-1}\hat{B}A^* = (\underline{c}, \underline{0})A^* = \underline{c}$$

9: Return the signature (s_2, r)

The main computational challenge is finding a lattice vector z close to t. Falcon solves this problem efficiently exploiting the recursive tree structure of $\hat{B}$ together with FFT representations, reducing the problem to operations over the integers through a divide-and-conquer approach.
Moreover, Gaussian sampling is used in order to randomize the signatures and prevent leakage of information about the private basis.

Verification

1: Check that $||s_2|| \leq \beta$ for some parameter β
2: Compute $s_1 + s_2 h = c \mod q$ where $c = H(m|r)$
3: Return 1 if $s_1 + s_2 h = c \mod q$, 0 otherwise

7.2.7 Comparison

Scheme	Security Level	Public Key	Secret Key	Signature Size
RSA-15360	$\approx$ 256 bits	$\approx$ 2 KB	$\approx$ 5 KB	$\approx$ 2 KB
ECDSA-521	$\approx$ 256 bits	$\approx$ 0.1 KB	$\approx$ 0.04 KB	$\approx$ 0.1 KB
Classic McEliece	$\approx$ 256 bits	$\approx$ 1.2 MB	$\approx$ 14 KB	N/A
HQC	$\approx$ 256 bits	$\approx$ 7 KB	$\approx$ 7 KB	N/A
Falcon-1024	$\approx$ 256 bits	$\approx$ 1.8 KB	$\approx$ 2.3 KB	$\approx$ 1.3 KB

Table 5: Approximate comparison of key and signature sizes for classical and post-quantum cryptographic schemes at the 256-bit security level.

In Falcon, signatures are represented by short polynomials sampled according to a discrete Gaussian distribution.
Although a coefficient in $\mathbb{Z}_q$ would normally require approximately $\log_2(q)$ bits, most sampled coefficients are concentrated around zero.
Indeed, for a Gaussian distribution with mean μ and standard deviation σ, approximately 99.7% of the values lie in the interval

$$[\mu - 3\sigma, \mu + 3\sigma].$$

Therefore, Falcon can compress signatures efficiently by encoding mainly small coefficients rather than arbitrary elements modulo q.
This allows Falcon to achieve significantly smaller signatures compared to many other post-quantum schemes.

8 Introduction to Multivariate Systems

NIST opened a call for signature-scheme proposals. The competition is currently in its third round, and nine finalists have been selected.
Four of the nine finalists are based on multivariate systems:

- MAYO
- UOV
- QR-UOV
- SNOVA

Recap: classes of complexity

42: P problems

The class **P** is the class of decision problems that are decidable in polynomial time by a deterministic Turing machine.

In other words, a problem is in **P** if there exists an algorithm that can solve any instance of the problem in time that is polynomial with respect to the size of the input.

43: NP problems

The class **NP** is the class of decision problems for which a given solution can be verified in polynomial time by a deterministic Turing machine.

In other words, a problem is in **NP** if there exists an algorithm that can verify whether a given solution to the problem is correct in polynomial time with respect to the size of the input.

44: NP-hard problems

A problem is **NP-hard** if every problem in NP can be reduced to it in polynomial time.

In other words, a problem is **NP-hard** if it is at least as hard as the hardest problems in NP. If a problem is NP-hard, it means that if we could solve it

efficiently, we could solve all problems in NP efficiently as well.

45: NP-complete problems

A problem is **NP-complete** if it is both in NP and NP-hard.

In other words, a problem is **NP-complete** if it is in NP and is as hard as the hardest problems in NP. If a problem is NP-complete, it means that if we could solve it efficiently, we could solve all problems in NP efficiently as well.

The question of whether $\mathbf{P} = \mathbf{NP}$ is one of the most important open problems in computer science.
It asks whether every problem that can be verified in polynomial time can also be solved in polynomial time. If $\mathbf{P} = \mathbf{NP}$, it would mean that all problems in NP can be solved efficiently, which would have profound implications for fields such as cryptography, optimization, and algorithm design. However, most experts believe that $\mathbf{P} \neq \mathbf{NP}$, meaning that there are problems in NP that cannot be solved efficiently, even though their solutions can be verified efficiently.

Many post-quantum cryptographic schemes are based on NP-complete problems.
Under specific assumptions, Multivariate Polynomial Problems are NP-complete.

8.1 Multivariate Polynomial Problems

46: Multivariate Polynomial Problem (MPP)

Given m polynomials of degree $d \geq 2$, $p_1, \ldots, p_m \in \mathbb{F}_q[x_1, \ldots, x_n]$, find a solution of the system

$$\begin{cases} p_1(x_1, \ldots, x_n) & = 0 \\ p_2(x_1, \ldots, x_n) & = 0 \\ \vdots & \\ p_m(x_1, \ldots, x_n) & = 0 \end{cases}$$

In other words, the Multivariate Polynomial Problem (MPP) asks us to find a solution to a system composed of m equations and n unknowns.

The particular case of the MPP where $m = 2$ and polynomials are of degree 2 is called the **MQ problem** (Multivariate Quadratic Problem).
The MQ problem is NP-complete, and it is the most common case in multivariate cryptography.

The choice to have only quadratic polynomials is motivated by the fact that solving a system of higher degree polynomials increase th complexity only polynomially.

8.2 Overview of Multivariate Cryptography

In multivariate cryptography, we can construcut both signature schemes and encryption schemes by exploiting the following idea:

Given the evaluation map

$$P : \mathbb{F}_q^n \to \mathbb{F}_q^m$$
$$x = (x_1, \ldots, x_n) \mapsto (p_1(x), \ldots, p_m(x))$$

where $p_1, \ldots, p_m$
and given $y = (y_1, \ldots, y_m)$, the difficult problem is to find a preimage $P^{-1}(y)$, i.e. a solution of the system, i.e an n-tuple $x = (x_1, \ldots, x_n)$ such that

$$P(x_1, \ldots, x_n) = (y_1, \ldots, y_m)$$

Note:
Let $f : A \to B$ be a function and $y \in B$. A preimage of y is an element $x \in A$ such that $f(x) = y$.

When we use this idea to construct a signature scheme we prefer to have $n < m$, so that the system is underdetermined and has many solutions.
On the other hand, when we use this idea to construct an encryption scheme we prefer to have $n = m$, so that the system is square and has a unique solution.

8.2.1 Central Map

The general framework consists of a secret fuction called central map, that is

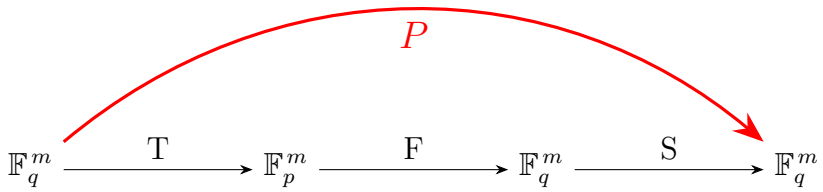
$$F : \mathbb{F}_q^n \to \mathbb{F}_q^m$$
$$x = (x_1, \ldots, x_n) \mapsto (f_1(x_1, \ldots, x_n), \ldots, f_m(x_1, \ldots, x_n))$$

and two secret affine transformations $S : \mathbb{F}_q^n \to \mathbb{F}_q^n$ and $T : \mathbb{F}_q^m \to \mathbb{F}_q^m$.
These two functions can also be rapresented as matrices, and we can write $S \in \mathbb{F}_q^{n \times n}$ and $T \in \mathbb{F}_q^{m \times m}$.

The trapdoor is the information of the internal structure of the map P, called the public map, where P is defined as:

$$P = S \circ F \circ T$$

That means starting from $x \in \mathbb{F}_q^n$, we first apply the transformation T to get $T(x)$, then we apply the central map F to get $F(T(x))$, and finally we apply the transformation S to get $P(x) = S(F(T(x)))$.



The main and most used trapdoors for multivariate cryptographic schemes are:

- Hidden Field Equations (HFE): the trapdoor information is that the public system of quadratic polynomials arises from a single (secret) univariate polynomial over a finite field extension $\mathbb{F}_{q^m}$.
- Oil and Vinegar (OV): the trapdoor information is that the system of polynomials vanishes on a small (secret) subspace, called the Oil subspace

Example of MQ problem

Let us consider the system of 4 multivariate polynomials in $\mathbb{F}_2[x_1, x_2, x_3]$.

$$\begin{cases} x_1x_3 + x_2x_3 + x_1 + x_2 + x_3 = 0, \\ x_2x_3 + x_2 + 1 = 0, \\ x_1x_2 + x_1x_3 + x_2 + x_3 = 0, \\ x_1x_2 + x_2x_3 + x_1 = 0. \end{cases}$$

We can ignore the fact that the system is quadratic and solve it as a linear system by introducing new variables y_i as:

$$y_1 = x_1, y_2 = x_2, y_3 = x_3,$$
$$y_4 = x_1x_2, y_5 = x_1x_3, y_6 = x_2x_3.$$

The system can be rewritten as:

$$\begin{cases} y_1 + y_3 + y_5 + y_6 = 0, \\ y_2 + y_6 + 1 = 0, \\ y_2 + y_3 + y_4 + y_5 = 0, \\ y_1 + y_4 + y_6 = 0. \end{cases}$$

The complete matrix of the system is:

$$\left(\begin{array}{cccccc|c} 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 1 & 0 \end{array}\right) \longrightarrow \left(\begin{array}{cccccc|c} 1 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array}\right)$$

so that reduced system is:

$$\begin{cases} y_1 = y_4 + y_6, \\ y_2 = y_6 + 1, \\ y_3 = y_4 + y_5 + y_6 + 1, \end{cases}$$

The 8 solutions $(y_1, y_2, y_3, y_4, y_5, y_6)$ of the linearized system are:

$$(0,1,1,0,0,0), \quad (1,0,0,0,0,1), \quad (0,1,0,0,1,0), \quad (1,0,1,0,1,1),$$
$$(1,1,0,1,0,0), \quad (0,0,1,1,0,1), \quad (1,1,1,1,1,0), \quad (0,0,0,1,1,1).$$

The problem is that they are not all solutions of the original system in the variables x_1, x_2, x_3.
The only actual solution is

$$(1,1,0,1,0,0).$$

For example, the solution

$$(y_1, y_2, y_3, y_4, y_5, y_6) = (0,1,1,0,0,0)$$

corresponds to:

$$x_2 = 1, \qquad x_3 = 1, \qquad x_2 x_3 = 0,$$

which is clearly not possible.

The step from the solution of the linearized system to the solution of the actual system gives rise, in general, to an exponential number of parasitic solutions.

8.2.2 Solving Multivariate Systems

In a quadratic system over the variables $x_1, \ldots, x_n$, linearization introduces one new variable y_i for each quadratic monomial. The number of quadratic monomials in n variables is

$$t = \frac{n(n+1)}{2}.$$

More generally, the number of monomials of degree d in n variables is

$$\binom{n+d-1}{d}.$$

After linearization, the original quadratic system is transformed into a linear system in t variables. If the system contains r linearly independent equations, then by linear algebra the number of free variables is

$$t - r.$$

Over a finite field $\mathbb{F}_q$, each free variable can assume q possible values. Therefore, the linearized system has

$$q^{t-r}$$

solutions.

In the quadratic case, since

$$t = \frac{n(n+1)}{2},$$

the number of solutions of the linearized system is

$$q^{\frac{n(n+1)}{2}-r}.$$

The key issue is that many of these solutions are not solutions of the original quadratic system. They are called *parasitic solutions*.

Usually, the number of actual solutions of the original system is small, while the number of solutions introduced by linearization can be exponential.

Thus, effective methods for solving MQ instances try to increase the number r of independent equations, i.e., the rank of the linearized system, in order to reduce

$$q^{t-r}$$

and therefore reduce the number of parasitic solutions.

8.2.3 Overview of Encryption and Signature

Multivariate cryptosystems are based on polynomial maps over a finite field. The public key is usually a system of multivariate polynomials

$$P = (p_1, \ldots, p_m),$$

which maps an input vector

$$M = (M_1, \ldots, M_n)$$

to an output vector

$$P(M) = (C_1, \ldots, C_m).$$

In an encryption setting, Alice computes

$$(C_1, \ldots, C_m) = P(M_1, \ldots, M_n)$$

and sends the ciphertext $C = (C_1, \ldots, C_m)$ to Bob.
Bob can efficiently recover the original message because he knows a secret trapdoor that allows him to solve

$$P(x_1, \ldots, x_n) = (C_1, \ldots, C_m).$$

Without this trapdoor, solving the corresponding multivariate polynomial system is assumed to be hard.

For digital signatures, a signature scheme is described by three algorithms:

- **KeyGeneration**(ε): given a security parameter ε, it generates a private key k and a public key P.
- **Signature**(M, k): the message M is signed using the private key k, producing a signature s.
- **Verification**(M, s, P): using the public key P, anyone can verify whether s is a valid signature for M.

In the multivariate setting, signing can be interpreted as finding a preimage of the hash of the message under the public polynomial map.
More precisely, given a message M, we first compute

$$H(M).$$

Then, using the secret trapdoor information, the signer efficiently finds a vector

$$s \in \mathbb{F}_q^m \text{ s. t. } P(s) = H(M)$$

The pair (M, s) is sent as the signed message.

Verification consists in checking whether

$$P(s) = H(M).$$

Finding such a preimage

$$s = P^{-1}(H(M))$$

without the secret trapdoor is assumed to be computationally hard.

9 Multivariate Cryptography

There are several crypto schemes based on multivariate idea.
One of the most important one is the **Oil and Vinegar** scheme, which is a signature scheme that made the basis for the most important signature schemes in the multivariate world.

9.1 Oil and Vinegar

It was proposed by Patarin in 1997.

The idea is to create a trapdoor for the central map of a multivariate quadratic scheme by separating the variables $x_1, \ldots, x_n$ in two different sets of variables:

- The vinegar variables $x_1, \ldots, x_v$;
- The oil variables $x_{v+1}, \ldots, x_{v+o}$.

There are v vinegar variables and o oil variables and a total of $n = o + v$ variables.
The original choice of Patarin was $o = v$, i.e. the same number of oil and vinegar variables, and this is now referred as **Balanced Oil and Vinegar**.

9.1.1 Central Maps

For semplicity, let us define two sets:

$$V = \{1, \ldots, v\} \ \textit{Vinegar variables}$$
$$O = \{v+1, \ldots, n\} \ \textit{Oil variables}$$

where $n = o + v$.

The *secret* central map is the system of polynomials:

$$F = (f_1(x_1, \ldots, x_n), \ldots, f_o(x_1, \ldots, x_n))$$

where each polynomial f_i has the form:

$$f_i(x) = \sum_{j,k \in V} a_{j,k}^{(i)} x_j x_k + \sum_{\substack{j \in V \\ k \in O}} b_{j,k}^{(i)} x_j x_k + \sum_{j \in V \cup O} c_j^{(i)} x_j + d^{(i)},$$

for all $i = 1, \ldots, o$ and for $x = (x_1, \ldots, x_n)$.

Note:
There is no product of two oil variables in the central map, which is the key point of the scheme, because if we fix the vinegar variables, the system becomes linear in the oil variables and can be easily solved.
In fact, the polynomials $f_i(x_1, \ldots, x_n)$, for $i = 1, \ldots, o$ have only:

- Linear terms in x_j, with $j = 1, \ldots, n$;
- product of two vinegar variables $x_j x_k$, with $j, k = 1, \ldots, v$;
- product of one vinegar variable and one oil variable $x_j x_k$, with $j = 1, \ldots, v$ and $k = v + 1, \ldots, n$.

So, the central map is a fuction:

$$F : \mathbb{F}_q^{v+o} \to \mathbb{F}_q^o$$

formed by o polynomials of degree 2 in n variables.

The *public* multivariate quadratic map is:

$$P = F \circ T$$

where T is a secret invertible affine transformation map of $\mathbb{F}_q$

In this case, we use only the map T, without the map S, because any linear transformation of OV polynomials is still an OV polynomial.
Even if we use a map S, it would be useless, because the structure of the central map is preserved by any linear transformation, so it would not add any security to the scheme.

9.1.2 key generation

1: We define the o polynomials $f_i(x_1, \ldots, x_n)$, $i = 1, \ldots, o$, as in (1) by choosing the coefficients $a_{j,k}^{(i)}$, $b_{j,k}^{(i)}$, $c_j^{(i)}$ and $d^{(i)}$ uniformly at random in $\mathbb{F}_q$.
2: We choose uniformly at random an affine invertible map $T : \mathbb{F}_q^n \to \mathbb{F}_q^n$ (or a matrix $T \in \mathbb{F}_q^{n \times n}$).

- **Public key:** The composition $P = F \circ T$, that is a quadratic polynomial system
- **Private key:** The two maps $T \in GL_n(\mathbb{F}_q)$ and $F = (f_1, \ldots, f_o)$.

9.1.3 Invert F

The knowledge of the central map $F = (f_1, \ldots, f_o)$, i.e. of the coefficients $a_{j,k}^{(i)}, b_{j,k}^{(i)}, c_j^{(i)}, c_k^{(i)}, d^{(i)}$, allows us to invert F efficiently.

Given an element of the image space $(t_1, \ldots, t_o) \in \mathbb{F}_q^o$, we want to find $(x_1, \ldots, x_n)$ such that

$$F(x_1, \ldots, x_n) = (t_1, \ldots, t_o).$$

This is done as follows. First, we randomly fix the vinegar variables

$$(x_1, \ldots, x_v) = (r_1, \ldots, r_v) \in \mathbb{F}_q^v.$$

Then, for $i = 1, \ldots, o$, we solve the system

$$f_i(r_1, \ldots, r_v, x_{v+1}, \ldots, x_{v+o}) = t_i.$$

After fixing the vinegar variables, the system becomes a linear system in o equations and o unknowns, namely the oil variables $x_{v+1}, \ldots, x_{v+o}$. Therefore, it can be solved efficiently.

There is a small probability that the random choice of $(r_1, \ldots, r_v)$ gives a linear system with no solution. In this case, we simply choose new vinegar values $(s_1, \ldots, s_v)$ and repeat the procedure.

A linear system with o equations and o unknowns has a unique solution whenever its coefficient matrix has full rank, equivalently when its determinant is non-zero.
Since most randomly obtained matrices are expected to be invertible, the procedure usually succeeds after few attempts.

The reason why this works is the special structure of the central map. Before fixing the vinegar variables, each polynomial has the form

$$f_i(x) = \sum_{j,k \in V} a_{j,k}^{(i)} x_j x_k + \sum_{j \in V,\ k \in O} b_{j,k}^{(i)} x_j x_k + \sum_{j \in V} c_j^{(i)} x_j + \sum_{k \in O} c_k^{(i)} x_k + d^{(i)}.$$

After substituting $(x_1, \ldots, x_v) = (r_1, \ldots, r_v)$, all terms involving only vinegar variables become constants, while mixed vinegar-oil terms become linear terms in the oil variables:

$$f_i(x) = \sum_{j,k \in V} a_{j,k}^{(i)} r_j r_k + \sum_{j \in V,\ k \in O} b_{j,k}^{(i)} r_j x_k + \sum_{j \in V} c_j^{(i)} r_j + \sum_{k \in O} c_k^{(i)} x_k + d^{(i)}.$$

Thus, the original quadratic system is transformed into a linear system in the oil variables, which can be solved in polynomial time.

9.1.4 Signature Generation

1: compute the hash $w = H(d) \in \mathbb{F}_q^o$ of the document d;
2: choose v random values $r_1, \ldots, r_v \in \mathbb{F}_q$ to assign to the vinegar variables;
3: solve the linear system $F(x_{v+1}, \ldots, x_{v+o}) = w$, finding the solution $(\ell_1, \ldots, \ell_o)$;
4: compute the signature $(z_1, \ldots, z_n) = T^{-1}(\ell_1, \ldots, \ell_n)$.

9.1.5 Signature verification

1: compute the hash $w = H(d)$;
2: compute $w' = P(z_1, \ldots, z_n) = F(T(z_1, \ldots, z_n))$ using the public system and the signature;
3: verify that $w = w'$;

9.1.6 Example

Let us consider an example over $\mathbb{F}_2$, with two oil variables o_1, o_2 and two vinegar variables v_1, v_2. The secret central map is

$$F : \mathbb{F}_2^4 \to \mathbb{F}_2^2,$$

defined as

$$(v_1, v_2, o_1, o_2) \mapsto \Big(f_1(v_1, v_2, o_1, o_2), f_2(v_1, v_2, o_1, o_2)\Big).$$

We choose the following quadratic polynomials:

$$f_1(v_1, v_2, o_1, o_2) = v_2 + o_1 v_2 + o_2 v_1 + v_1 v_2,$$

$$f_2(v_1, v_2, o_1, o_2) = o_2 + o_2 v_2 + o_1 v_1 + 1.$$

These are Oil and Vinegar polynomials, because there are no quadratic terms containing only oil variables.

The polynomial system containing f_1 and f_2 is easy to invert. Indeed, for any

$$(a_1, a_2) \in \mathbb{F}_2^2,$$

we can solve

$$\begin{cases} f_1(v_1, v_2, o_1, o_2) = a_1, \\ f_2(v_1, v_2, o_1, o_2) = a_2, \end{cases}$$

by assigning random values to the vinegar variables. Once the vinegar variables are fixed, the system becomes linear in the oil variables.

To hide the special Oil and Vinegar structure, we mix the input with an invertible affine map

$$T : \mathbb{F}_2^4 \to \mathbb{F}_2^4.$$

In this example, we use the matrix

$$T = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix}.$$

The input transforms as

$$T \begin{pmatrix} v_1 \\ v_2 \\ o_1 \\ o_2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \\ o_1 \\ o_2 \end{pmatrix} = \begin{pmatrix} v_1 + o_2 \\ o_2 \\ v_2 \\ o_1 + o_2 \end{pmatrix}.$$

Therefore, the public polynomials are obtained by composing F with T:

$$p_1(v_1, v_2, o_1, o_2) = f_1(T(v_1, v_2, o_1, o_2)),$$

$$p_2(v_1, v_2, o_1, o_2) = f_2(T(v_1, v_2, o_1, o_2)).$$

More explicitly:

$$p_1(v_1, v_2, o_1, o_2) = f_1(v_1 + o_2, o_2, v_2, o_1 + o_2) = o_2 + o_1o_2 + o_2v_2 + o_1v_1,$$

$$p_2(v_1, v_2, o_1, o_2) = f_2(v_1 + o_2, o_2, v_2, o_1 + o_2) = o_1 + o_1o_2 + v_2v_1 + o_2v_2 + 1.$$

Hence, the public system is

$$\begin{cases} p_1(v_1, v_2, o_1, o_2) = o_2 + o_1o_2 + o_2v_2 + o_1v_1, \\ p_2(v_1, v_2, o_1, o_2) = o_1 + o_1o_2 + v_2v_1 + o_2v_2 + 1. \end{cases}$$

This public system contains both quadratic terms in the vinegar variables and quadratic terms in the oil variables. Therefore, the public polynomials are no longer Oil and Vinegar polynomials. This makes the system hard to

invert for someone who does not know the secret transformation T and the private polynomials f_1, f_2.

Now suppose we want to sign the message

$$M = (0, 1) \in \mathbb{F}_2^2.$$

A signature is a vector

$$s = (v_1, v_2, o_1, o_2) \in \mathbb{F}_2^4$$

such that

$$P(s) = M,$$

where $P = (p_1, p_2)$ is the public map.
Equivalently, we need to solve

$$\begin{cases} o_2 + o_1 o_2 + o_2 v_2 + o_1 v_1 = 0, \\ o_1 + o_1 o_2 + v_2 v_1 + o_2 v_2 + 1 = 1. \end{cases}$$

Since we know the secret key, we solve this in two steps:

1. find (v_1', v_2', o_1', o_2') such that

$$F(v_1', v_2', o_1', o_2') = (0, 1);$$

2. find (v_1, v_2, o_1, o_2) such that

$$T(v_1, v_2, o_1, o_2) = (v_1', v_2', o_1', o_2').$$

We first solve

$$\begin{cases} f_1(v_1', v_2', o_1', o_2') = 0, \\ f_2(v_1', v_2', o_1', o_2') = 1. \end{cases}$$

Using the structure of the private polynomials, this becomes

$$\begin{cases} v_2' + o_1' v_2' + o_2' v_1' + v_1' v_2' = 0, \\ o_2' + o_2' v_2' + o_1' v_1' = 0. \end{cases}$$

We assign random values to the vinegar variables. For example, choose

$$(v_1', v_2') = (1, 1).$$

Substituting these values, we obtain a linear system in the oil variables:

$$\begin{cases} o_1' + o_2' = 0, \\ o_1' = 0. \end{cases}$$

Thus,

$$(o_1', o_2') = (0, 0),$$

and therefore

$$(v_1', v_2', o_1', o_2') = (1, 1, 0, 0).$$

Now we recover the original variables by applying T^{-1}:

$$\begin{pmatrix} v_1 \\ v_2 \\ o_1 \\ o_2 \end{pmatrix} = T^{-1} \begin{pmatrix} v_1' \\ v_2' \\ o_1' \\ o_2' \end{pmatrix} = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \\ 1 \end{pmatrix}.$$

Therefore, the signature for the message $M = (0, 1)$ is

$$s = (0, 0, 1, 1).$$

The pair (M, s) is published. Anyone can verify the signature using the public polynomials:

$$\begin{cases} p_1(0, 0, 1, 1) = 1 + 1 + 0 + 0 = 0, \\ p_2(0, 0, 1, 1) = 1 + 1 + 0 + 0 + 1 = 1. \end{cases}$$

Therefore,

$$P(s) = M.$$

Thus, anyone can verify the validity of the signature, but producing such a signature without knowing the secret key (T, f_1, f_2) is computationally hard.

9.1.7 Security of the scheme

Patarin's original choice is $o = v$. In this case, both the secret and the public maps are functions

$$F : \mathbb{F}_q^{2o} \to \mathbb{F}_q^o.$$

The balanced version of Oil and Vinegar, where $o = v$, is therefore not considered secure becauss it was attacked by Kipnis and Shamir in 1998.

For this reason, in 1999, Kipnis, Patarin and Goubin introduced the *Unbalanced Oil and Vinegar* scheme, usually denoted as UOV, where

$$v > o.$$

However, if v is too large, approximately when

$$v > \frac{o^2}{2},$$

the scheme also becomes insecure, because there are many more indeterminates than equations.

A commonly considered secure choice of parameters satisfies

$$o < v < 3o.$$

An efficient choice is obtained by taking approximately

$$v \approx 1.35o.$$

9.2 Kipnis Attack

The all schemes based on the Oil and Vinegar idea are vulnerable to the attack of Kipnis and Shamir, which is a structural attack that exploits the special structure of the central map.

9.2.1 Idea of the attack

The attack of Kipnis and Shamir is based on the observation that the central map F has a special structure that can be exploited to recover the secret key.

The central map is defined as (using a $\mathbf{F}_2$ example):

$$f(v_1, v_2, o_1, o_2) = o_2 + o_1 + v_1o_1 + v_1v_2$$

can be mede homogeneous by adding a new variable v_3:

$$\hat{f}(v_1, v_2, v_3, o_1, o_2) = v_3o_2 + v_3o_1 + v_1o_1 + v_1v_2$$

Let $M = (m_1, .., m_k)$ be a message consisting of k elements from a finite field $\mathcal{F}$ of order q.
$X = (x_1, .., x_{2k})$ consists of $2k$ elements from $\mathcal{F}$, is a valid signature for M if satisfies $G(X) = M$, where $G(x) : \mathcal{F}^{2k} \to \mathcal{F}^k$ is the public map of the scheme (public key).

The mapping G can be written as $G(X) = (G_1(X), \ldots, G_k(X))$, where each $G_e(X)$ is a homogeneous quadratic form of $2k$ variables $X = (x_1, \ldots, x_{2k})$ over $\mathcal{F}$.

Such quatradic forms can be represented by the product $X^T G_e X$ where G_e is a symmetric matrix $2k \times 2k$ and X is a column vector of $2k$ variables.

Example:
Let consider the quadratic form in the private nonlinear system by matrice $F_e = \left(\begin{array}{c|c} 0 & B_1 \\ \hline B_2 & B_3 \end{array}\right)$ Consider the following example:

$$F_e = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix} \text{ and } Y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix}$$

Then, the quadratic form is:

$$\begin{aligned} Y^T F_e Y &= \begin{pmatrix} y_1 & y_2 & y_3 & y_4 \end{pmatrix} \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} \\ &= \begin{pmatrix} y_3 + y_4, & 0 & y_1 + y_3 & y_1 + y_2 + y_4 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \end{pmatrix} \\ &= y_1 y_3 + y_1 y_4 + y_1 y_3 + y_3^2 + y_1 y_4 + y_2 y_4 + y_4^2 \end{aligned}$$

In **red** I underlined that we don't have the quadratic terms y_1 and y_2 due the zero block in the upper left corner of the matrix F_e.
Thus, when we multiply by Y, the variable y_1 and y_2 are not multiplied by themselves, i.e. they are Oil variables.

In the context of OV schemes, we have that each message M has approximately q^k valid signatures.
The fact that the upper-left block of the matrices defining the central quadratic map is zero implies that the quadratic forms do not contain products between two oil variables.
In other words, in a monomial of the form $c_{ij} y_i y_j$, at most one of the two indices i and j can refer to an oil variable. As a consequence, the oil variables appear only linearly in the central equations, while the vinegar variables can appear either linearly or quadratically.

This distinction is the key idea behind the Oil and Vinegar construction. The central map is easy to invert for the legitimate signer because the signer

knows which variables are oil variables and which variables are vinegar variables.
However, this structure is hidden in the public key by the secret affine transformation. When the central equations are expressed in terms of the public variables X, the separation between oil and vinegar variables disappears, and the public equations look like random quadratic equations in which all variables seem to interact with each other through random-looking coefficients.

To sign a message

$$M = (m_1, \ldots, m_k),$$

the legitimate signer first assigns random values to all vinegar variables,

$$y_{k+1}, \ldots, y_{2k}.$$

Once these values are fixed, the central quadratic equations

$$Y^t F_e Y = m_e$$

are simplified. Since there are no quadratic terms involving two oil variables, the resulting equations become a system of k linear equations in the k oil variables. This linear system can be solved efficiently.

If the system is singular, the signer chooses new random vinegar values and repeats the procedure. Otherwise, after solving the linear system, the signer obtains a complete internal solution Y, containing both the randomly chosen vinegar variables and the computed oil variables. This solution is then mapped back to the public variables using the inverse of the secret affine transformation,

$$X = A^{-1}Y.$$

The vector X is finally returned as the signature of the message M.

From the point of view of an attacker, the situation is different. The attacker only sees the public quadratic equations

$$G_i(X) = m_i, \qquad i = 1, \ldots, k,$$

and does not know the hidden oil–vinegar separation. Therefore, forging a signature seems to require solving a system of random-looking multivariate quadratic equations.

However, Kipnis and Shamir showed that the scheme can be broken by recovering the hidden oil variables, or more precisely the oil subspace. The important observation is that the public quadratic forms still preserve some algebraic information about this hidden subspace. In particular, the oil subspace is a common invariant subspace for a family of matrices derived from the public key.

The attack constructs matrices of the form

$$G_i^{-1} G_j,$$

starting from the public quadratic forms. Although the secret affine transformation hides the original oil and vinegar variables, these derived matrices preserve the oil subspace. Therefore, by studying their common eigenspaces, the attacker can recover a subspace equivalent to the secret oil space.

Once this subspace is found, the attacker can choose a new basis in which the first k variables span the recovered oil subspace and the remaining variables complete the basis. In this new representation, the public quadratic equations recover the same useful property of the secret central map: they become linear in the oil variables once the vinegar variables are fixed.

At this point, the attacker can imitate the legitimate signing algorithm. He assigns random values to the vinegar variables, obtains a linear system in the oil variables, solves it, and maps the solution back to the public variables. In this way, the attacker can forge valid signatures for arbitrary messages, even without reconstructing the exact original secret key.

9.3 Unbalanced Oil and Vinegar

Unbalanced Oil and Vinegar (UOV) is a variant of the original Oil and Vinegar scheme that was introduced to address the security weaknesses of the balanced version.
In UOV, the number of vinegar variables v is greater than the number of oil variables o. This unbalance makes the scheme more resistant to structural attacks, such as the Kipnis-Shamir attack.

Moreover, UOV is designed to be more efficient than the original Oil and Vinegar, it is faster to sign and verify signatures, and it has reasoneble small signature sizes.
The real drawback of UOV is that the public key size is quite large, which can be a problem for some applications.

Hash and Sign

Given the public/secret key pair $(P, (T, F))$, it is straightforward to build the original UOV signature scheme using the hash-and-sign paradigm.

- The given message is first hashed to a target vector $t \in \mathbb{F}_q^m$ and the secret key $(T, F))$ enables us to efficiently find the premiage $s \in \mathbb{F}_q^n$ of t under the public map P, i.e. such that $P(s) = t$. The signature is the vector s.
- To verify a signature s for a message M, the verifier computes the hash $t = H(M)$ and checks that $P(s) = t$.

9.3.1 Overview of the scheme

The key generation algorithm of UOV is the same as the one of the original Oil and Vinegar scheme, with the only difference that we choose $v > o$.

Signature Generation

1: compute the hash $t = H(\mu)$, with $\mu \in \{0,1\}^*$
2: find a random premiage $u \in \mathbb{F}_q^n$ such that $\mathcal{F}(u) = t$ using the secret key $(\mathcal{F}, \mathcal{T})$
3: $s := \mathcal{T}^{-1}(u)$
4: $\sigma := s$
5: return σ as signature of μ

Signature verification

1: compute the hash $t = H(\mu)$
2: compute $t' = P(\sigma)$ using the public key
3: verify that $t = t'$

9.3.2 Central Map

The central map of UOV follows the same oil–vinegar principle described for the basic OV scheme. It is a quadratic map

$$\mathcal{F} : \mathbb{F}_q^n \to \mathbb{F}_q^m$$

where the first $v = n - m$ variables are vinegar variables and the remaining m variables are oil variables. Each polynomial is constructed so that no quadratic term contains the product of two oil variables:

$$f^{(k)}(x_1, \ldots, x_n) = \sum_{i=1}^{v} \sum_{j=i}^{n} \alpha_{i,j}^{(k)} x_i x_j.$$

As a consequence, fixing the vinegar variables turns the system into a linear system in the oil variables.

$$\mathcal{T} : \mathbb{F}_q^n \to \mathbb{F}_q^n$$

is an invertible linear transformation, which is used to hide the structure of the central map $\mathcal{F}$ in $\mathcal{P}$.

Inversion operation of $\mathcal{P}$

The inversion of the public map

$$\mathcal{P} = \mathcal{F} \circ \mathcal{T}$$

is efficient for the legitimate signer because both $\mathcal{F}$ and $\mathcal{T}$ are known. Since $\mathcal{T}$ is an invertible linear transformation, inverting $\mathcal{P}$ essentially reduces to finding a preimage under the central map $\mathcal{F}$.

In UOV, this is done by randomly fixing the vinegar variables. Once these variables are fixed, the quadratic system defined by $\mathcal{F}$ becomes a linear system in the oil variables. Therefore, the signer can solve it using Gaussian elimination.

If the resulting linear system has full rank, a valid preimage is obtained. If not, the signer simply chooses new random vinegar variables and repeats the procedure. Since the induced linear system is full-rank with high probability, only a polynomial number of attempts is needed.

Finally, once a preimage under $\mathcal{F}$ is found, the signer applies

$$s = \mathcal{T}^{-1} \begin{pmatrix} u \\ w \end{pmatrix}$$

to obtain a valid signature s for the target message.

9.3.3 Specifications

One of the most important contributions of authors is the idea behind the "compression".

$$\text{UOV's paramenters } (m = 96, n = 244, q = 256, v = n - m)$$

Secret key size

Racall that the secret key (in the classic schema) is given by the two maps:

1. $\mathcal{F} : \mathbb{F}_q^n \to \mathbb{F}_q^m$ (the central map that represents the nonlinear part of the scheme easy to solve)
2. $\mathcal{T} : \mathbb{F}_q^n \to \mathbb{F}_q^n$ (the linear transformation, *Professor called it "the map that chose the coordinates"*)

Since

$$\mathcal{F}$$

describes the non-linear system with m equations.
Remembering that a quadratic polynomial with m variables can be denoted by a matrix in $\mathbb{F}_q^{m \times m}$, we have that $\mathcal{F}$ is denoted by $m \cdot m^2$ elements of $\mathbb{F}_q$.

$$\mathcal{T}$$

describes a linear map, thus it can be represented by a matrix in $\mathbb{F}_q^{m \times m}$, we have that $\mathcal{T}$ is denoted by m^2 elements of $\mathbb{F}_q$.
Thus, the private key $(\mathcal{F}, \mathcal{T})$ has size

$$\log_2 q \cdot m^2 \cdot m + \log_2 q \cdot m \text{ bits}$$

$$\simeq 5.7 \text{ MB}.$$

In UOV, the trapdoor, i.e., the secret key is represented by a matrix

$$\begin{pmatrix} O \\ I_m \end{pmatrix}$$

where

$$O \in \mathbb{F}_q^{v \times m}.$$

Indeed, the Oil space for the map $\mathcal{F}$ is

$$\text{span}(e_{v+1}, \ldots, e_m)$$

where $e_1, \ldots, e_m$ are the vectors of the canonical basis.

Indeed, for $\mathcal{F}$ the Oil variables are the last m variables of

$$(x_1, x_2, \ldots, x_v, x_{v+1}, \ldots, x_m).$$

Then, the corresponding Oil space for the map $\mathcal{P}$ is

$$\text{span}\left(\mathcal{T}^{-1}(e_{v+1}), \ldots, \mathcal{T}^{-1}(e_m)\right),$$

which is a vector subspace of $\mathbb{F}_q^n$ with dimension m.

In general, a vector subspace of $\mathbb{F}_q^n$ with dimension m is described by a basis with m vectors, i.e. a standard way for constructing such vectors is to consider

$$\begin{aligned}
&(a_1, \ldots, a_v, 1, 0, \ldots, 0),\\
&(b_1, \ldots, b_v, 0, 1, \ldots, 0),\\
&\ldots,\\
&(m_1, \ldots, m_v, 0, 0, \ldots, 1).
\end{aligned}$$

which are surely linearly independent.
These vectors, where $a_1, \ldots, a_v$, $b_1, \ldots, b_v$, …, $m_1, \ldots, m_v$ can be randomly chosen in $\mathbb{F}_q$, are the columns of the matrix O.

Considering

$$O \in \mathbb{F}_q^{v \times m}$$

as private key, then its size is

$$\log_2 q \cdot v \cdot m$$

bits

$$\simeq 14.2 \text{ KB}.$$

Finally, since the entries of O are random, we can only memorize the seed for generating them and consequently the private key has only size 32 bytes.

Public key size

In the classical scheme is given by the map

$$\mathcal{P} : \mathbb{F}_q^m \longrightarrow \mathbb{F}_q^m$$

which represents the non-linear system hard to solve.
Similarly to $\mathcal{F}$, the map $\mathcal{P}$ can be described by

$$\log_2 q \cdot m \cdot m^2$$

bits

$$\simeq 5.6 \text{ MB}.$$

In UOV, the quadratic polynomials of $\mathcal{P}$ are described by m matrices

$$P_i = \begin{pmatrix} P_i^{(1)} & P_i^{(2)} \\ 0 & P_i^{(3)} \end{pmatrix}, \qquad i = 1, \ldots, m,$$

in $\mathbb{F}_q^{m \times m}$, where $P_i^{(1)}$ and $P_i^{(2)}$ are randomly computed, while $P_i^{(3)}$ is computed from $P_i^{(1)}$, $P_i^{(2)}$ and O.
Thus for describing the public key we need m matrices in $\mathbb{F}_q^{m \times m}$, i.e.,

$$\log_2 q \cdot m^3$$

bits

$$\simeq 884 \text{ KB}.$$

Actually, $P_i^{(3)}$, for $i = 1, \ldots, m$, are upper triangular matrices.
Thus, we can describe each of these matrices with

$$m + (m-1) + (m-2) + \cdots + 2 + 1 = \frac{m(m+1)}{2}$$

elements of $\mathbb{F}_q$.
Then, the public key is described by

$$\log_2 q \cdot m \cdot \frac{m(m+1)}{2}$$

bits

$$= 447 \text{ KB}.$$